

Средства симуляции ЦП и ОС

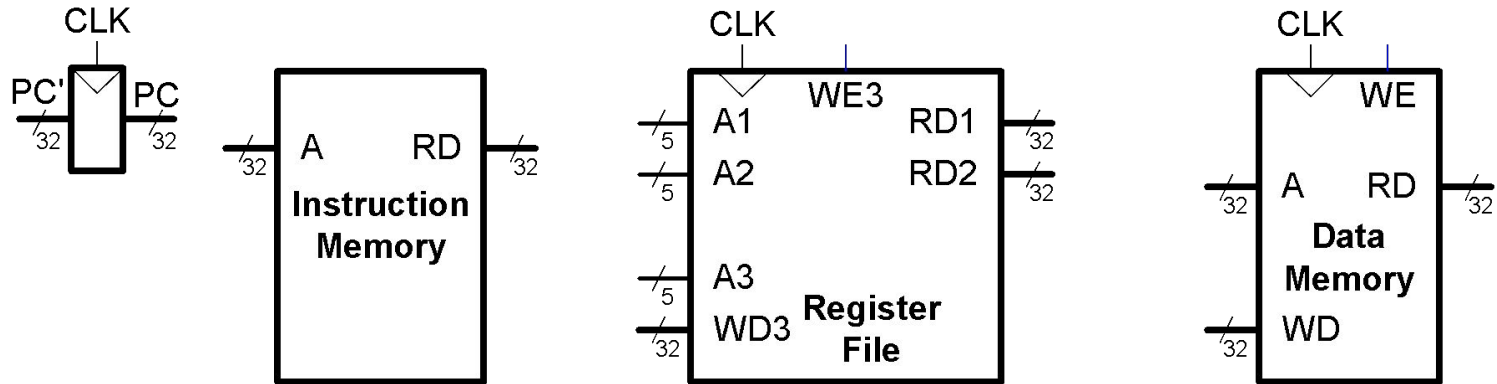
Лекция №16



Державин Андрей
Шурыгин Антон

Введение

- Ввели основные понятия для проектирования первого процессора:



Recap

- Производительность

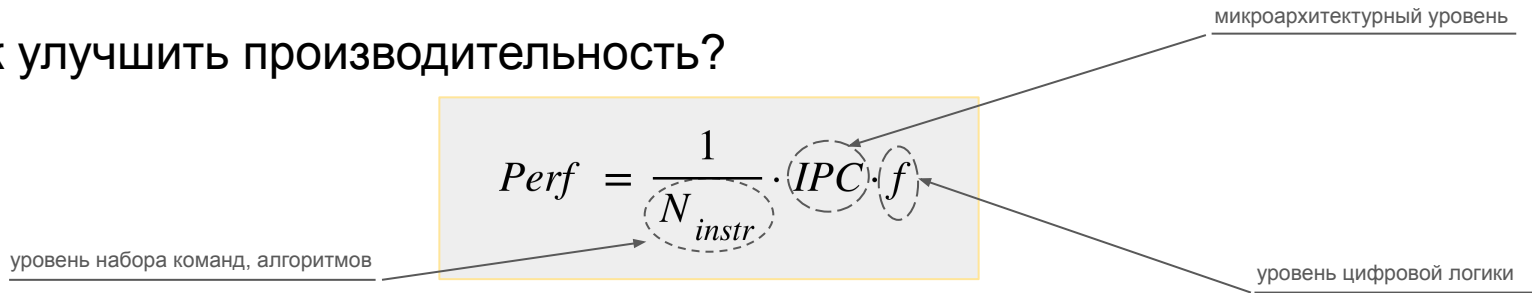
$$Perf = \frac{1}{Time} = \frac{1}{N_{instr} \cdot CPI \cdot T_{cycle}} = \frac{1}{N_{instr}} \cdot IPC \cdot f$$

- Время исполнения

$$Time = T_{cycle} \cdot Cycles\ Num = N_{instr} \cdot CPI \cdot T_{cycle}$$

1. **IPC** - среднее количество инструкций, выполняемых за каждый такт
2. **CPI** - величина, обратная IPC.
3. **f** - тактовая частота

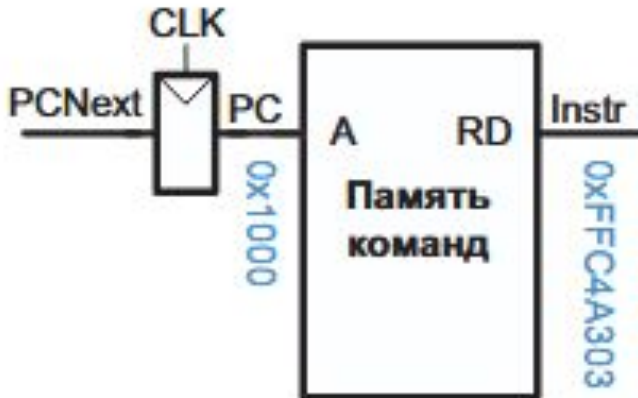
- Как улучшить производительность?



Single cycle CPU

- С чего начинается исполнение инструкции?
- С fetch!

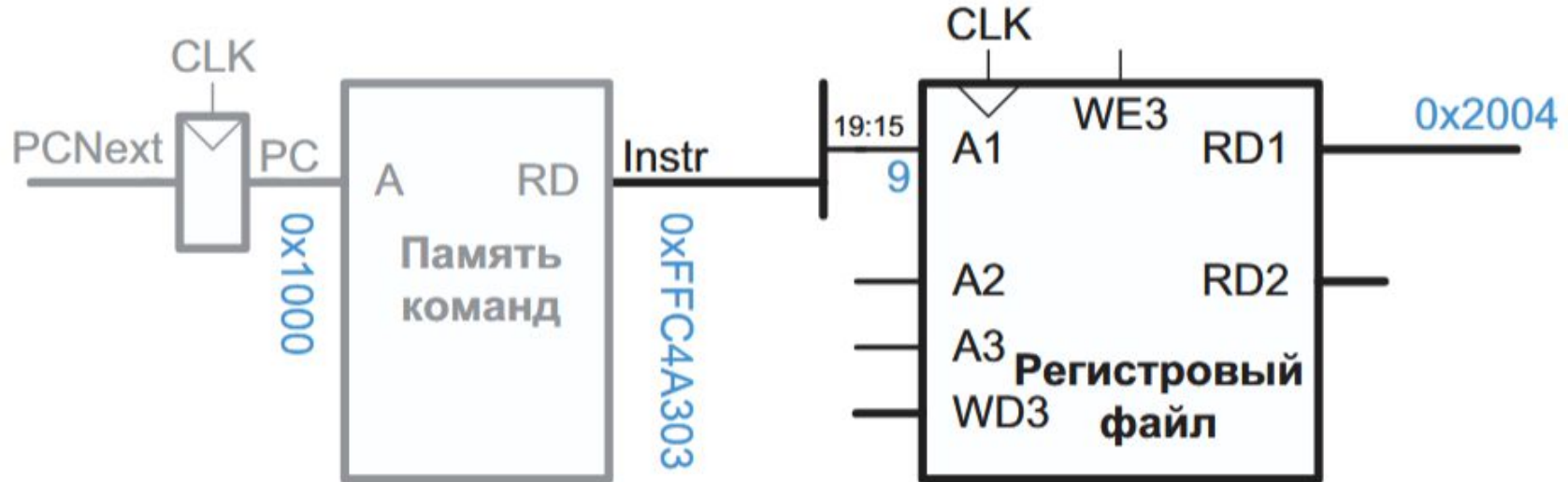
0x1000: lw x6, -4(x9) → 0xFFC4A303



Single cycle CPU

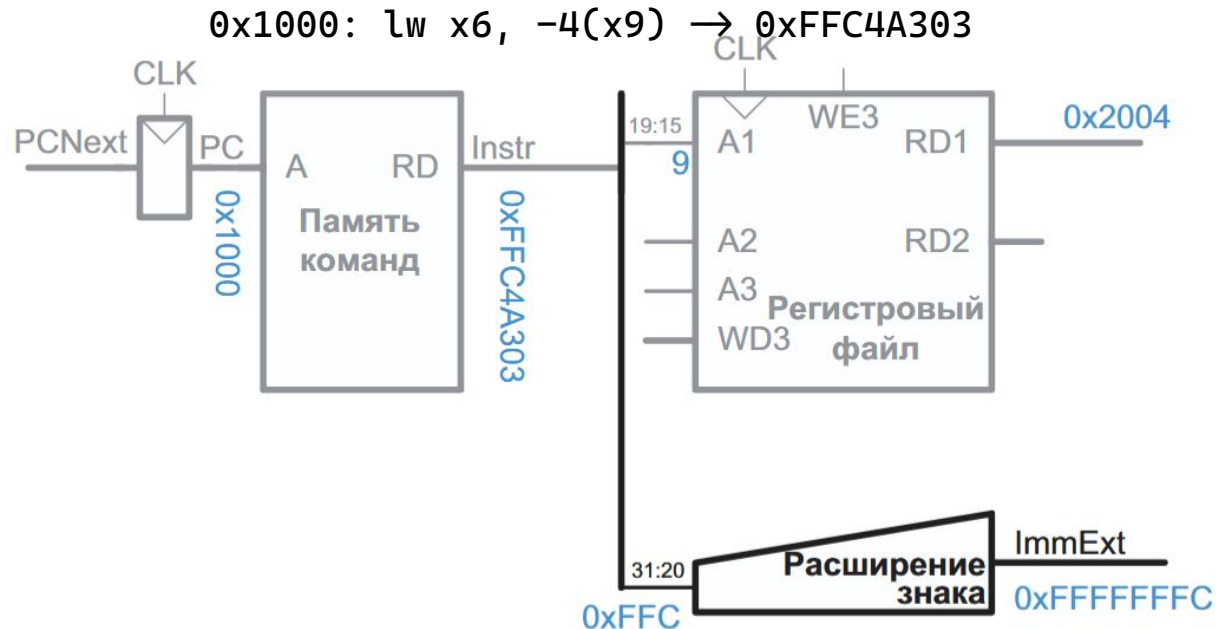
- Загрузили байты инструкции из памяти
- Надо декодировать

0x1000: lw x6, -4(x9) → 0xFFC4A303



Single cycle CPU

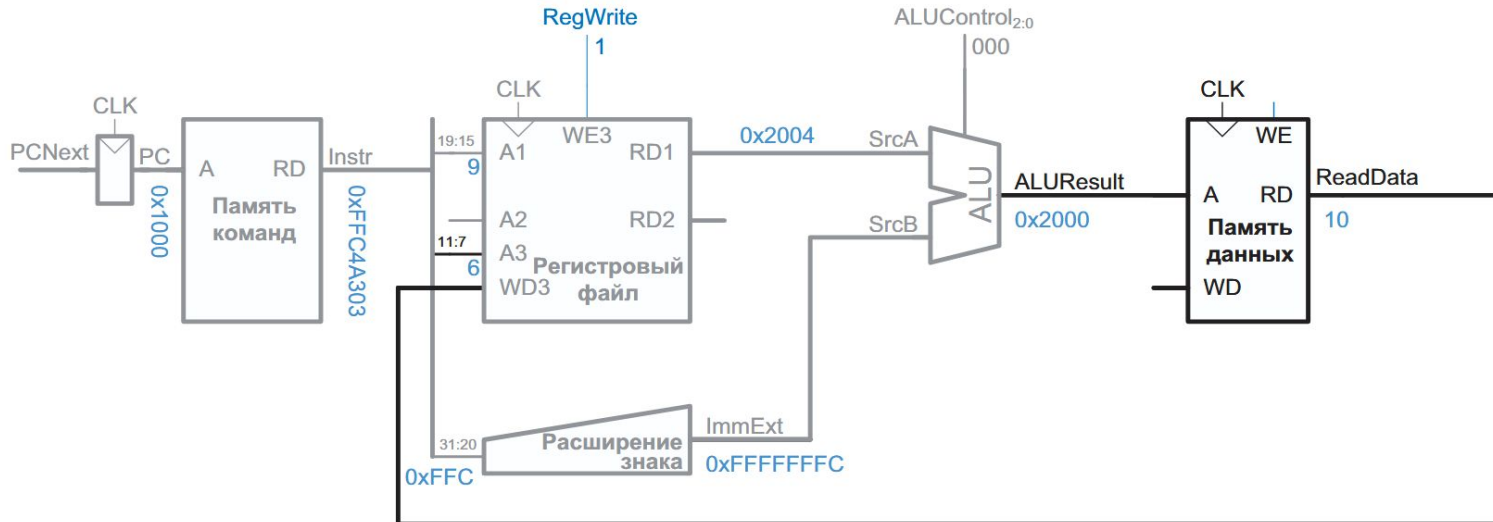
- Не забываем про знаковое расширение



Single cycle CPU

- Пора считать адрес
- И обращаться в память

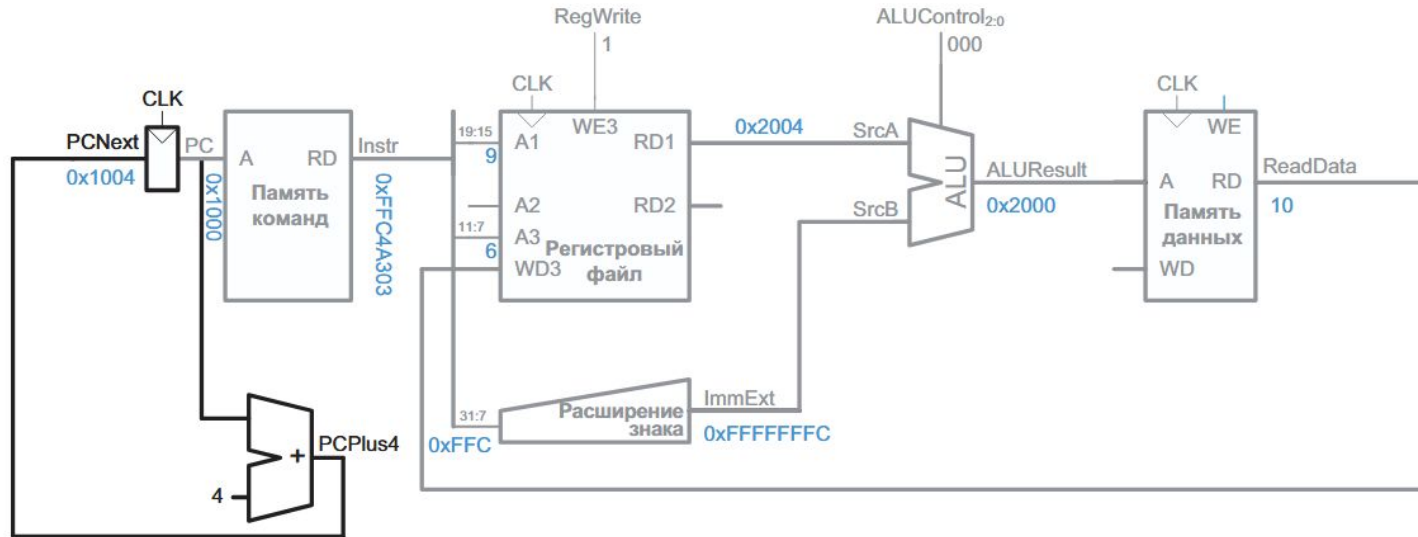
0x1000: lw x6, -4(x9) → 0xFFC4A303



Single cycle CPU

- Как исполнить больше одной инструкции?
- Не забываем обновить PC

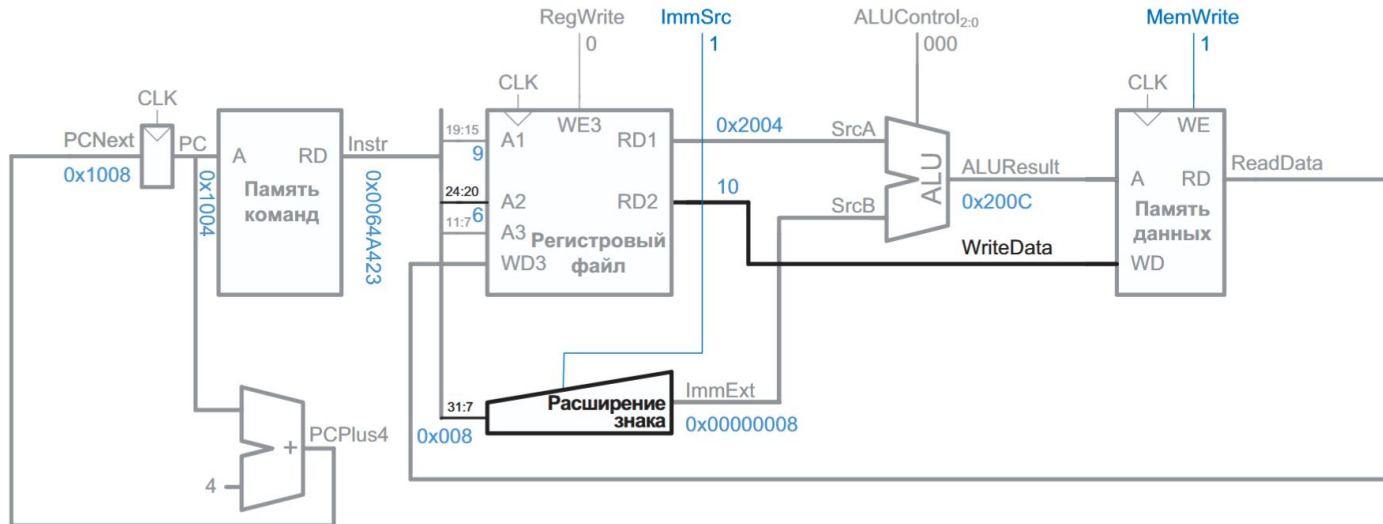
0x1000: lw x6, -4(x9) → 0xFFC4A303



Single cycle CPU

- Что добавится при поддержке sw?
- Запись в память

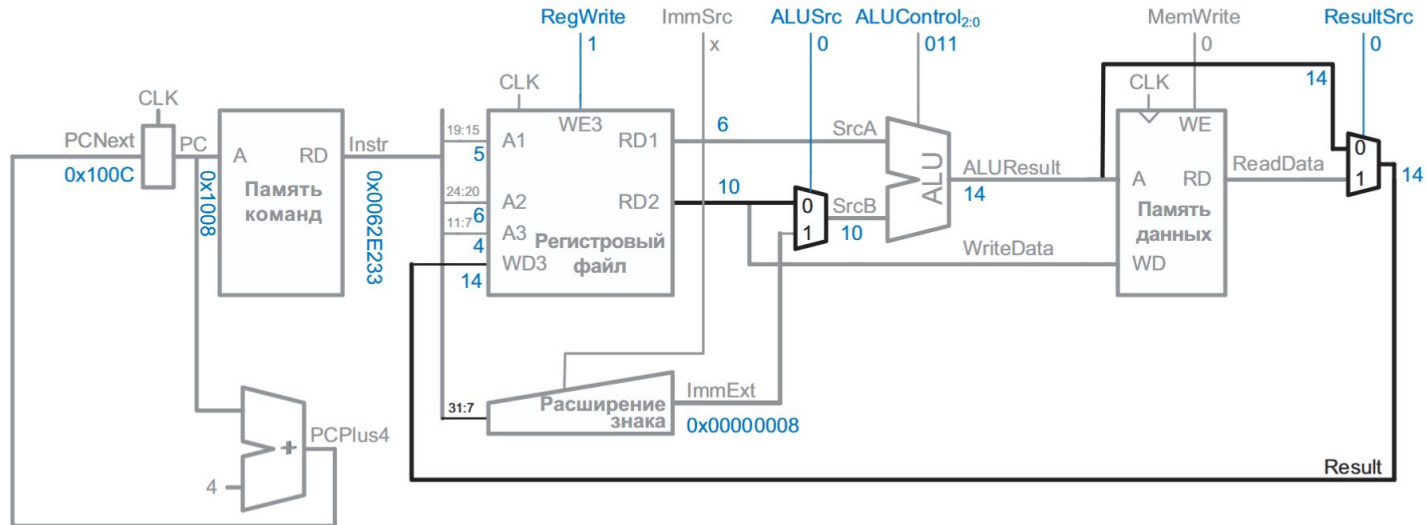
0x1004: sw x6, 8(x9) → 0x0064A423



Single cycle CPU

- Поддержим весь R-type

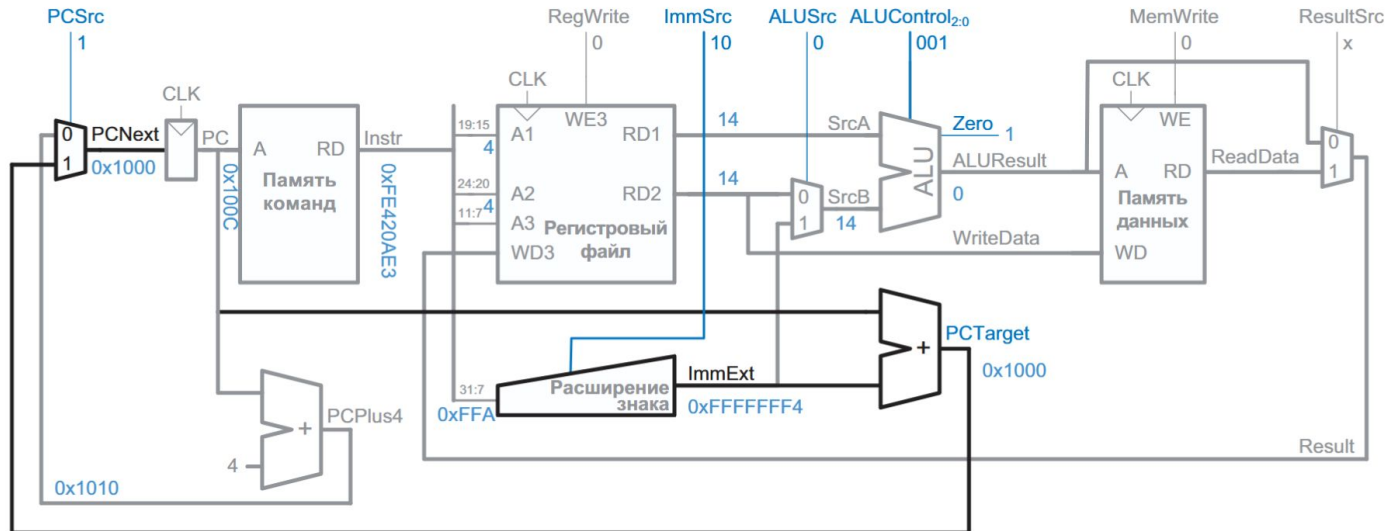
0x1008: or x4, x5, x6 → 0x0062E233



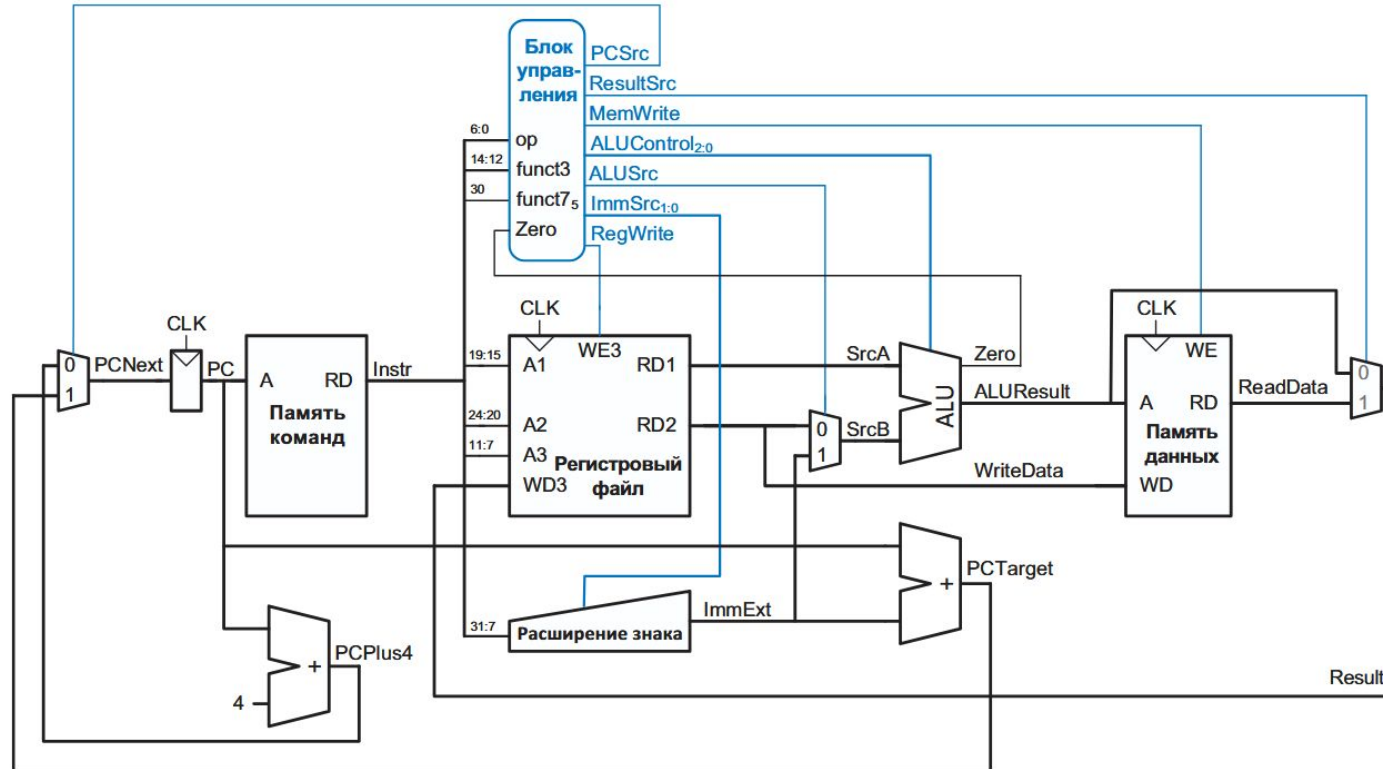
Single cycle CPU

- Пока что код исполняется линейно
- Добавим логику для branch

0x100C: beq x4, x4, 0x1000 → 0xFE420AE3

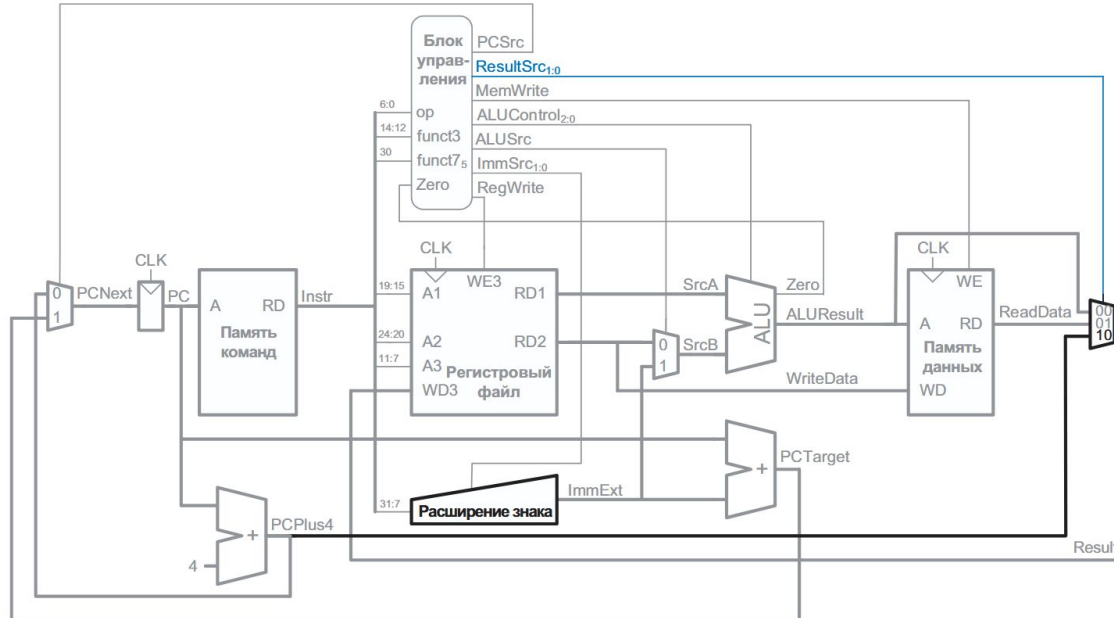


Single-cycle CPU



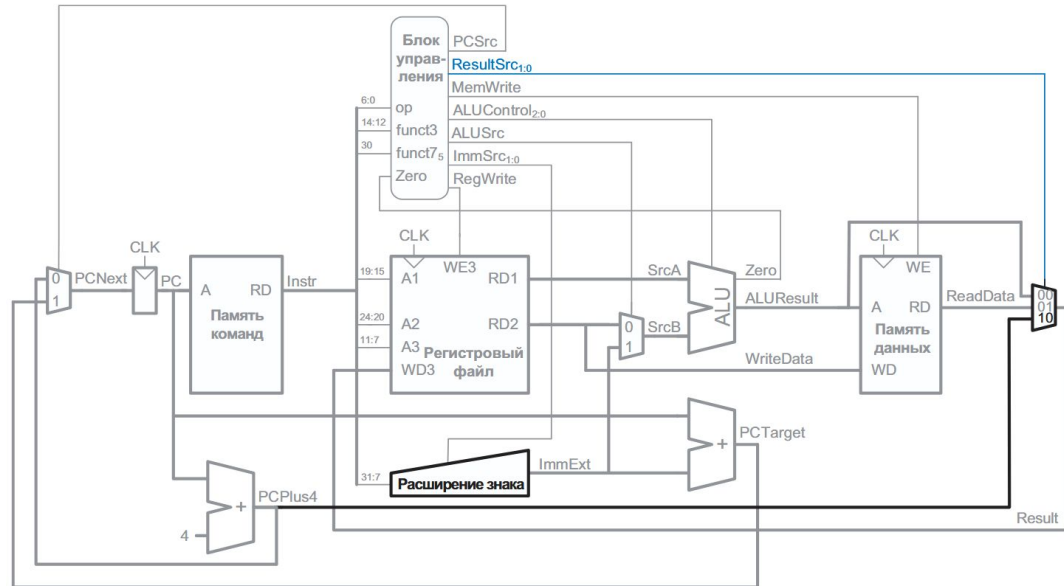
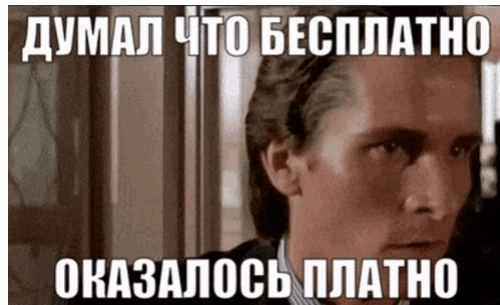
Single-cycle CPU

- Минимальные доработки для jal



Single-cycle CPU

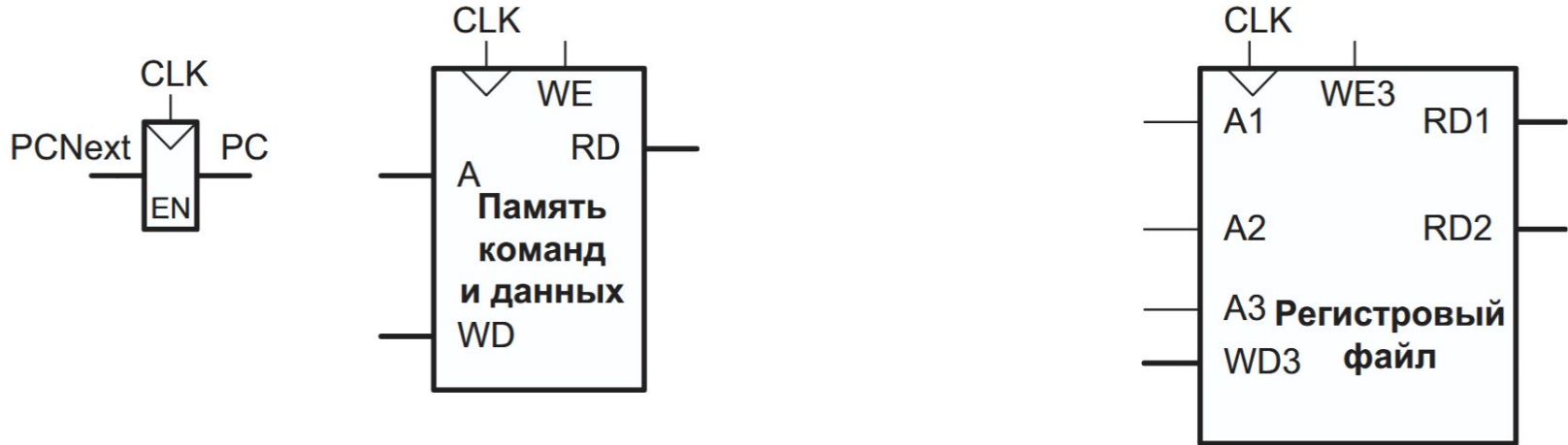
- +:
 - Простая модель
- -:
 - “Гарвардская архитектура” (?)
 - Период такта определяется длиной критического пути
 - Три сумматора



Multi-cycle CPU

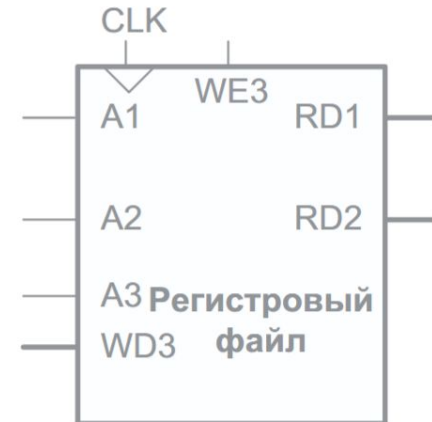
Multi-cycle CPU

- Попробуем решить одноктактные проблемы
 - Объединим память
 - Разобьём операции на шаги
 - Переиспользуем ресурсы



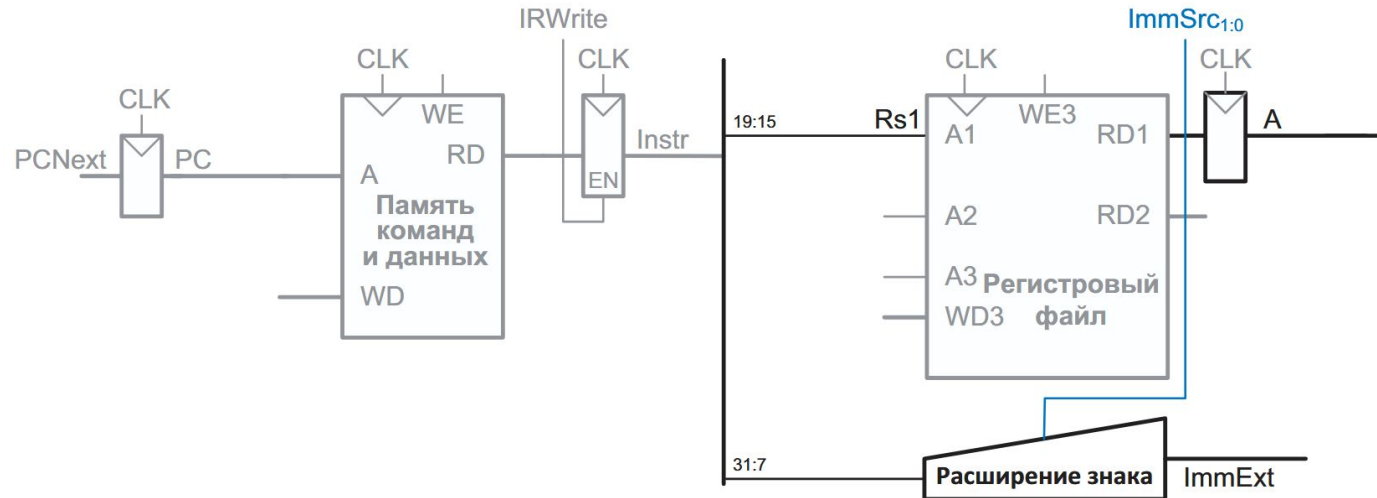
Multi-cycle CPU

- Снова читаем инструкцию
- Теперь инструкция хранится в регистре



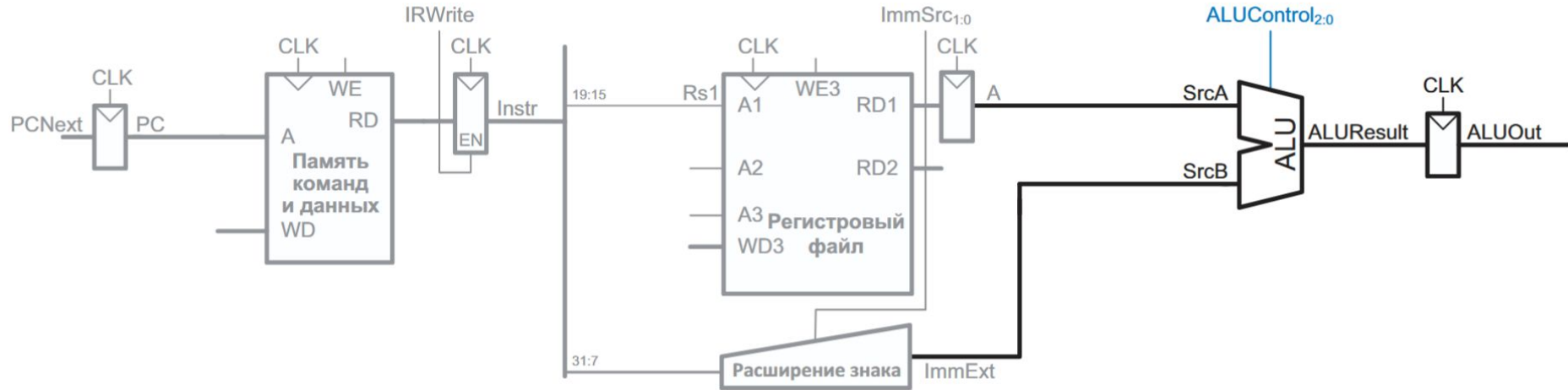
Multi-cycle CPU

- Попробуем реализовать lw
- Добавим чтение операндов



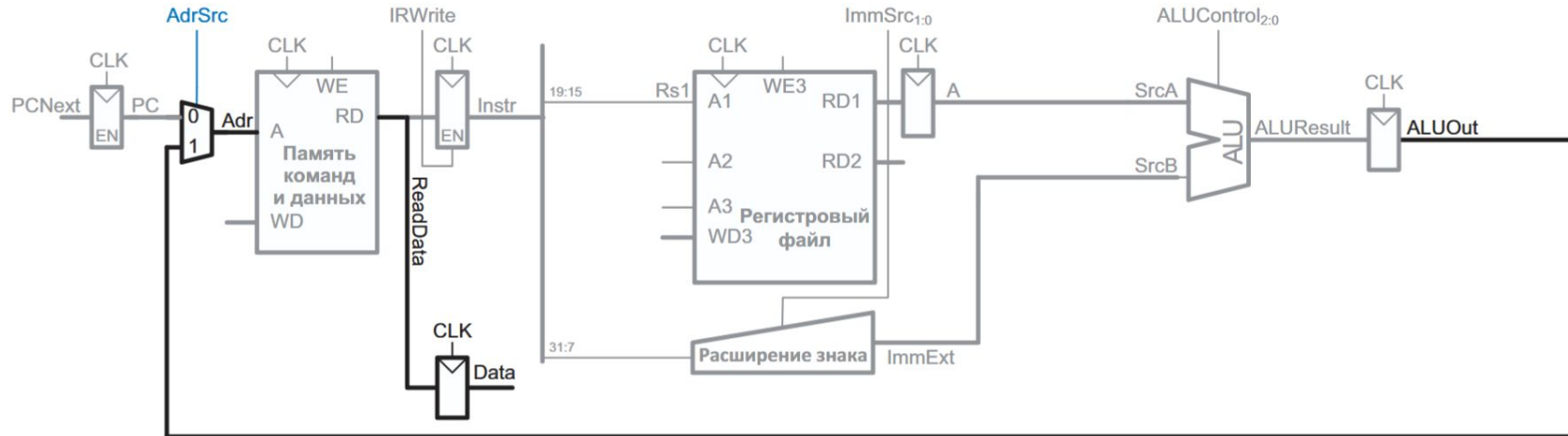
Multi-cycle CPU

- Попробуем реализовать lw
- Добавим подсчет адреса



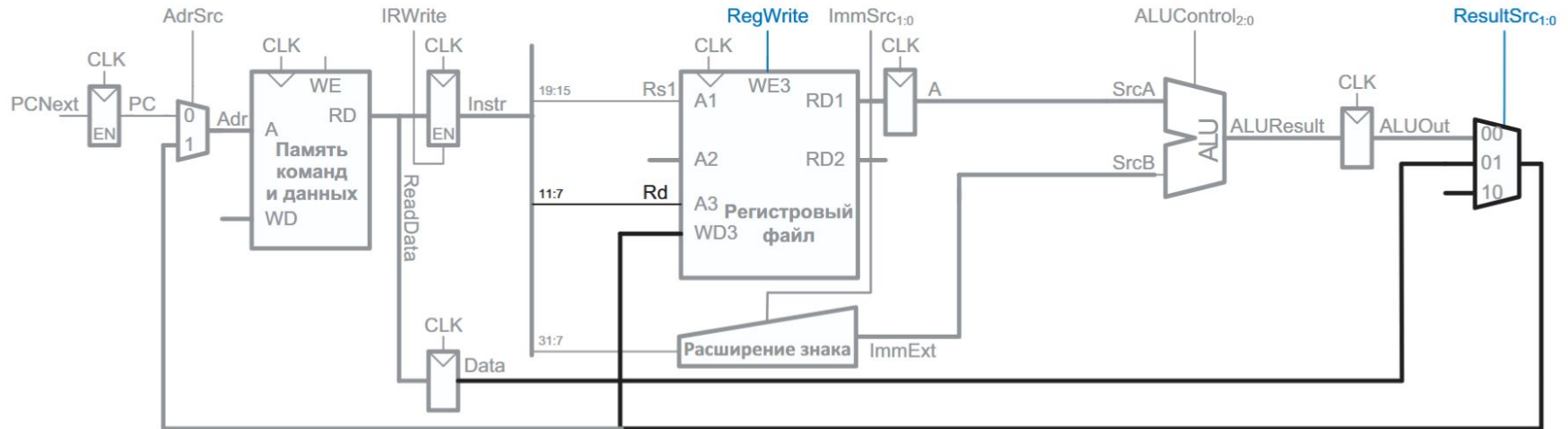
Multi-cycle CPU

- Попробуем реализовать lw
- Обратимся в память (ту же самую)



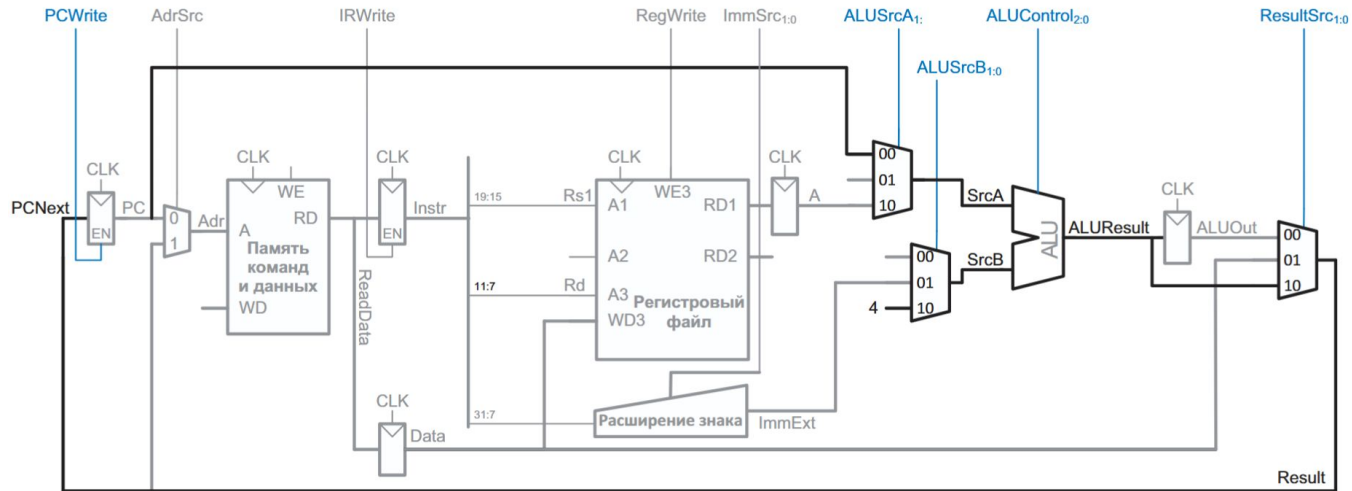
Multi-cycle CPU

- Попробуем реализовать lw
- Запишем результат в регистр



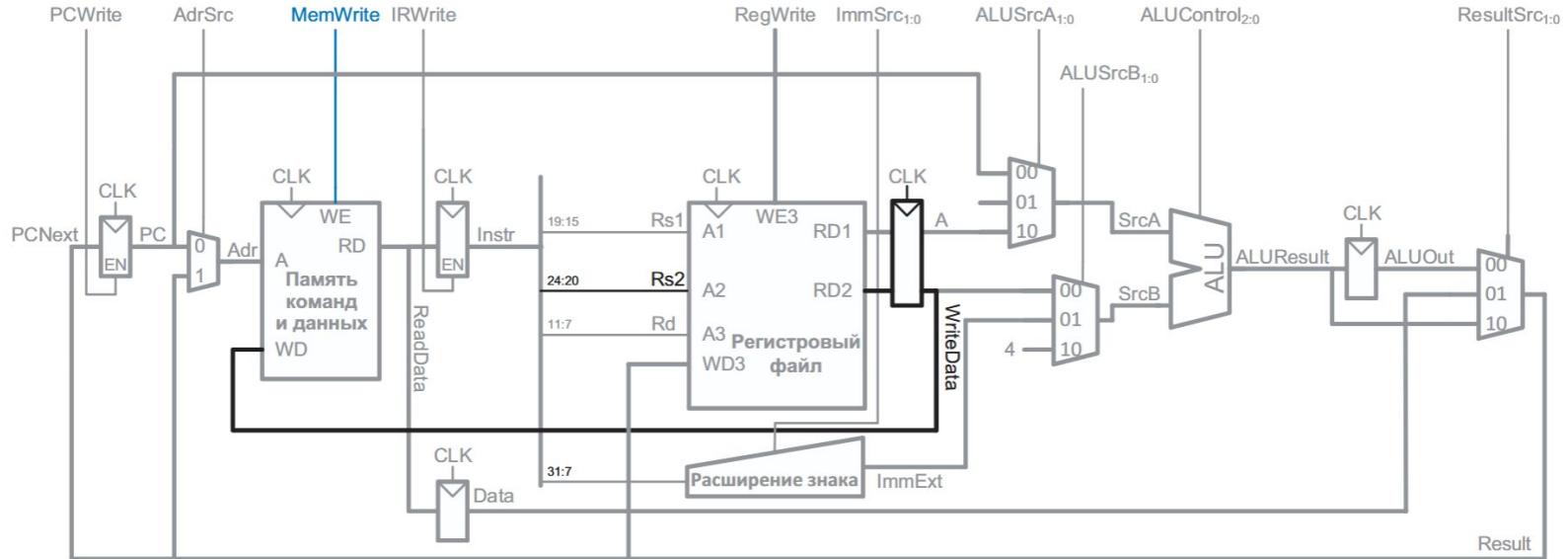
Multi-cycle CPU

- Попробуем реализовать lw
- Идем дальше



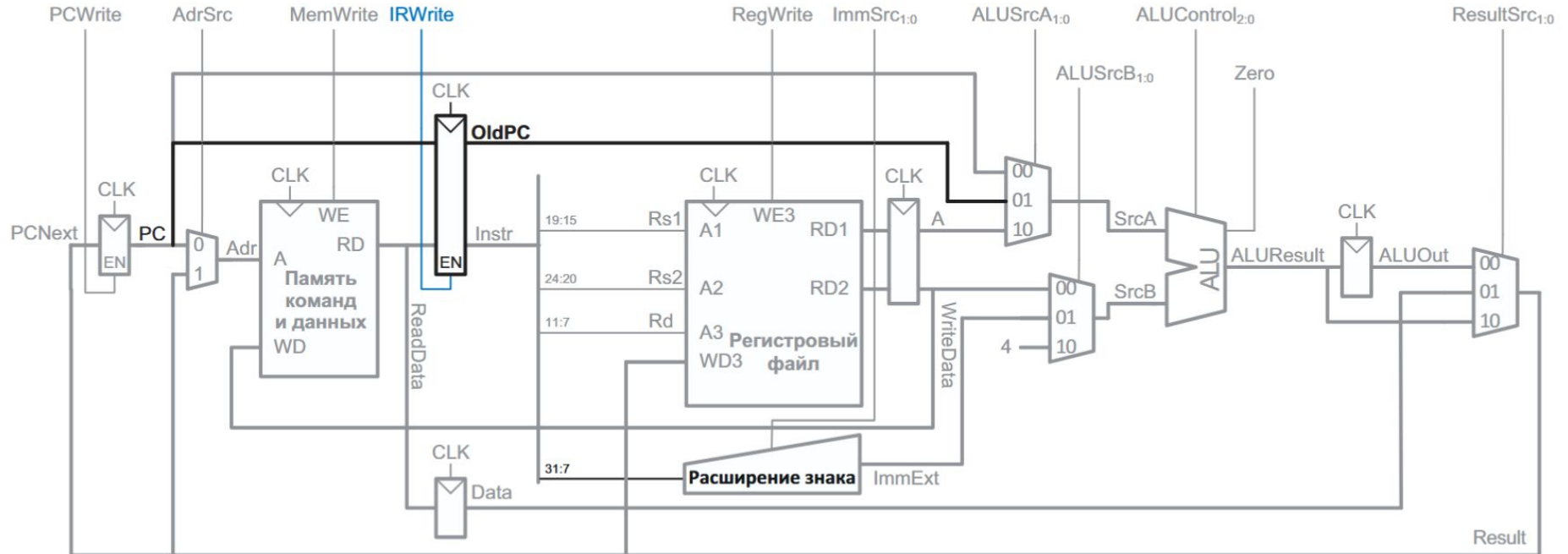
Multi-cycle CPU

- Снова доработаем до sw

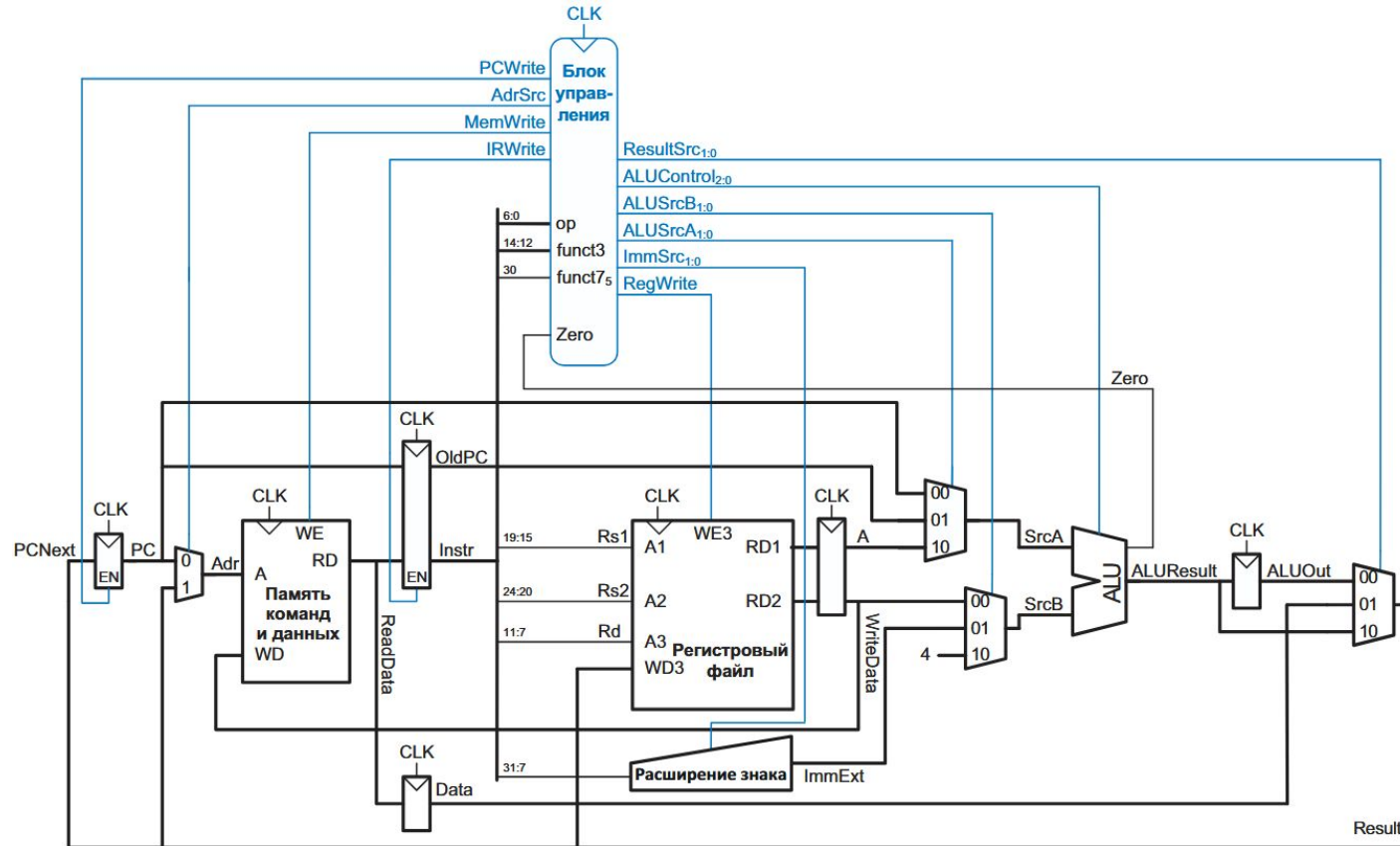


Multi-cycle CPU

- Помним про beq

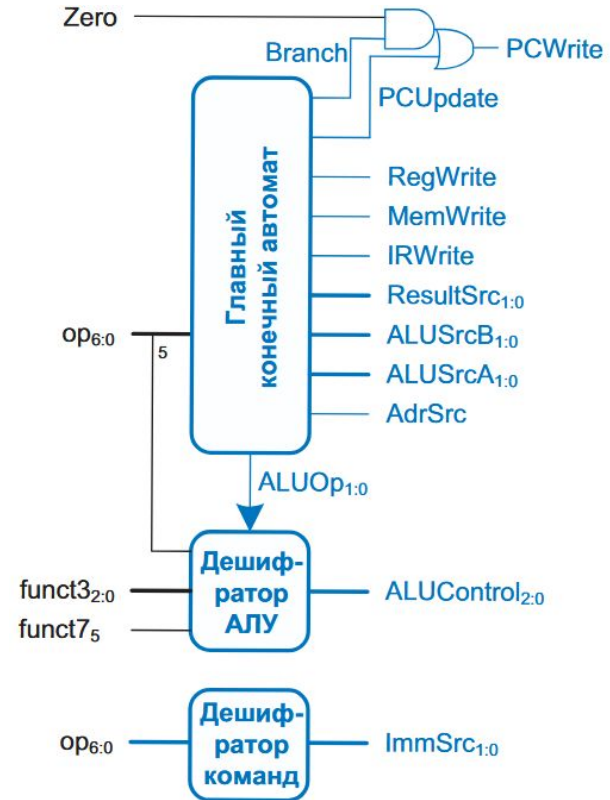


Multi-cycle CPU



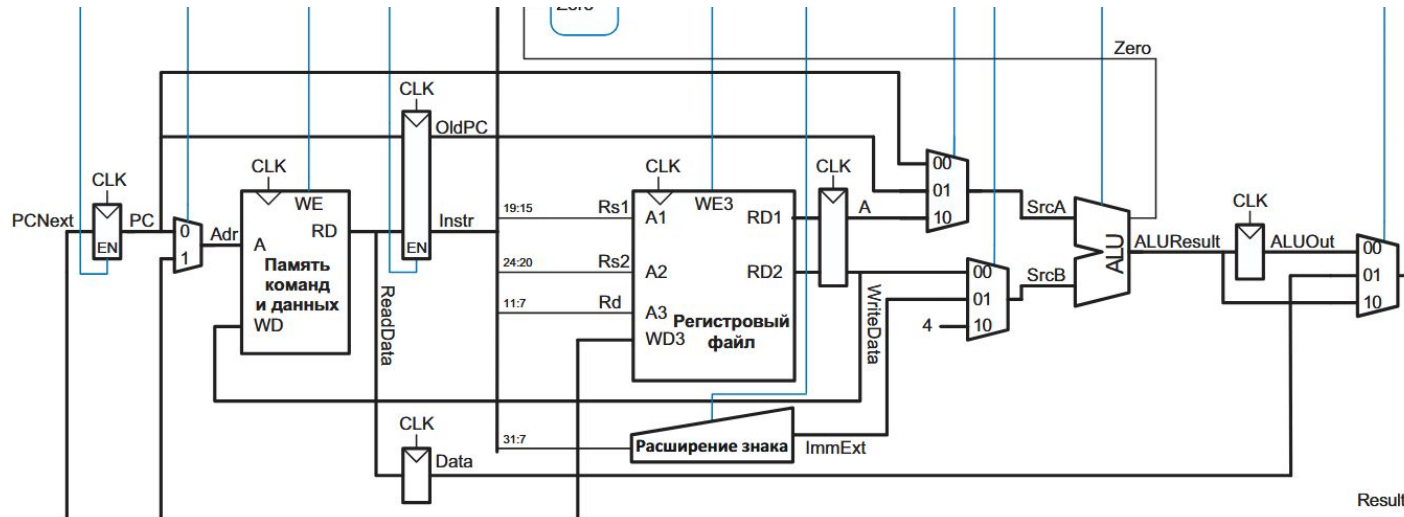
Multi-cycle CPU. Control block

- Главный конечный автомат
 - Формирует нужные сигналы в каждом такте
 - Основывается на предыдущем состоянии



Multi-cycle CPU. Performance

- Сколько тактов занимают инструкции?
 - beq - 3 такта
 - jal, R, l, sw - 4 такта
 - lw - 5 тактов



Multi-cycle CPU. Performance

- Сколько тактов занимают инструкции?
 - beq - 3 такта
 - jal, R, l, sw - 4 такта
 - lw - 5 тактов
- SPEC
 - 25% loads
 - 10% stores
 - 13% branches
 - 52% R-type
- $CPI = 0.13 * 3 + 0.62 * 4 + 0.25 * 5 = 4.12$
- Однако длительность такта не сильно уменьшилась
- Не все этапы имеют одну длину

Pipelined CPU

Pipelined CPU

**Башкортостанец собрал «Ламбу» в гараже:
итальянские заводы в панике**

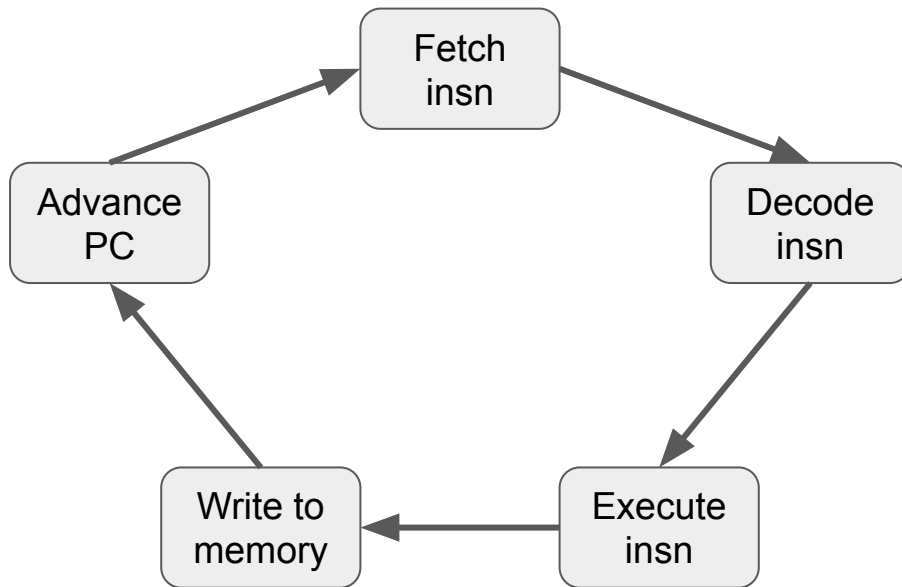
Non-pipelined



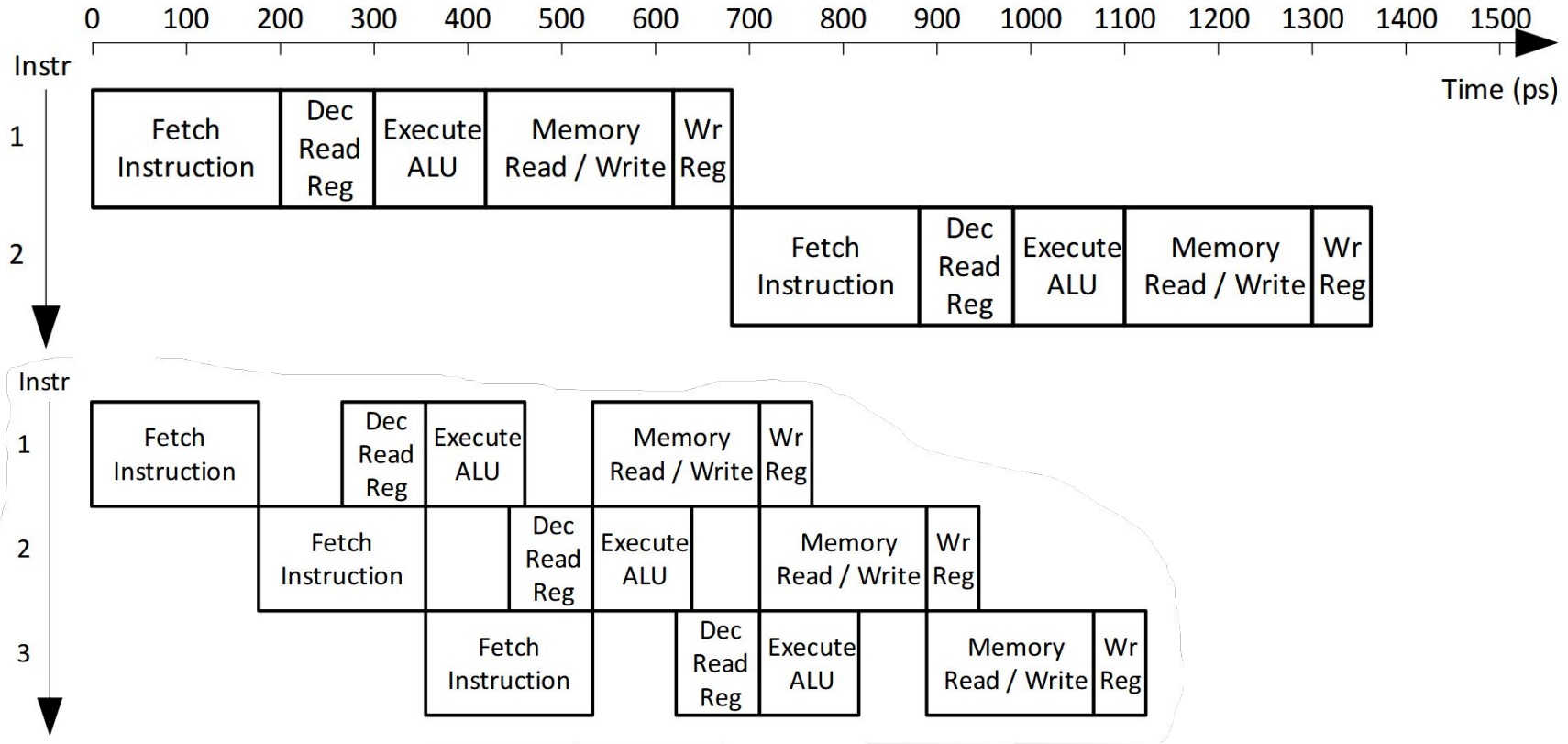
Pipelined

Pipelined CPU

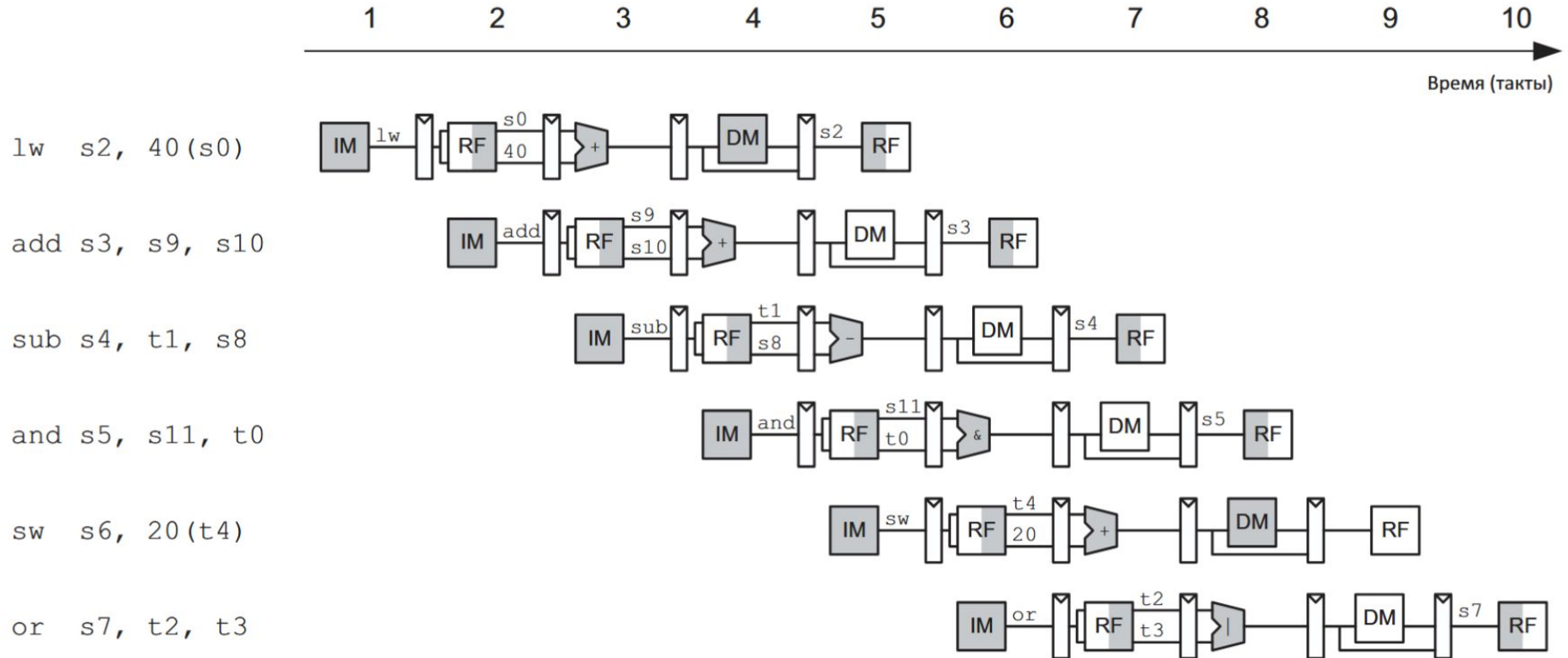
- Разделяем single cycle на стадии
- Повышаем утилизацию компонент
- Каждая стадия включает одну “долгую” операцию



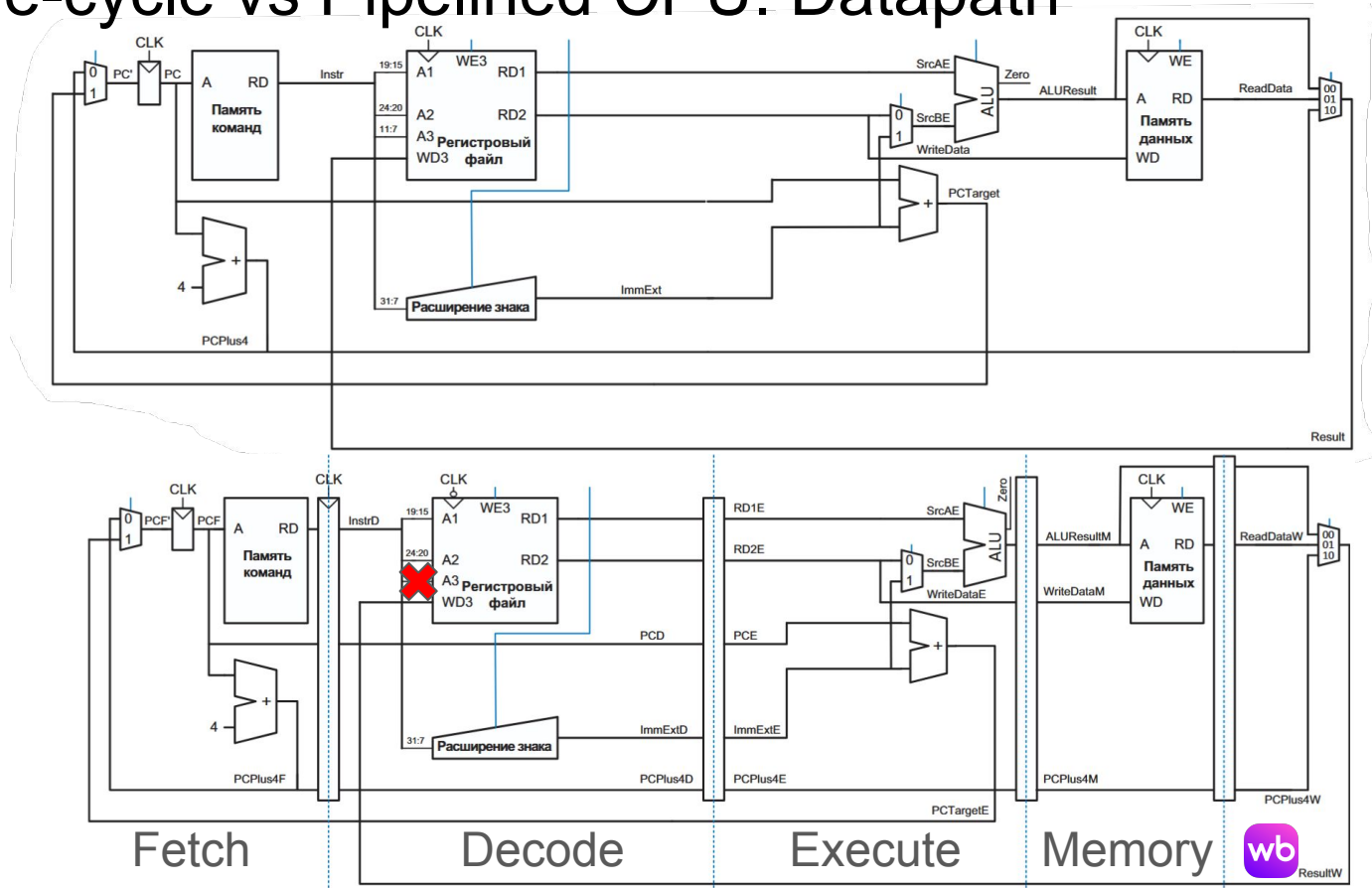
Single-cycle vs Pipelined CPU



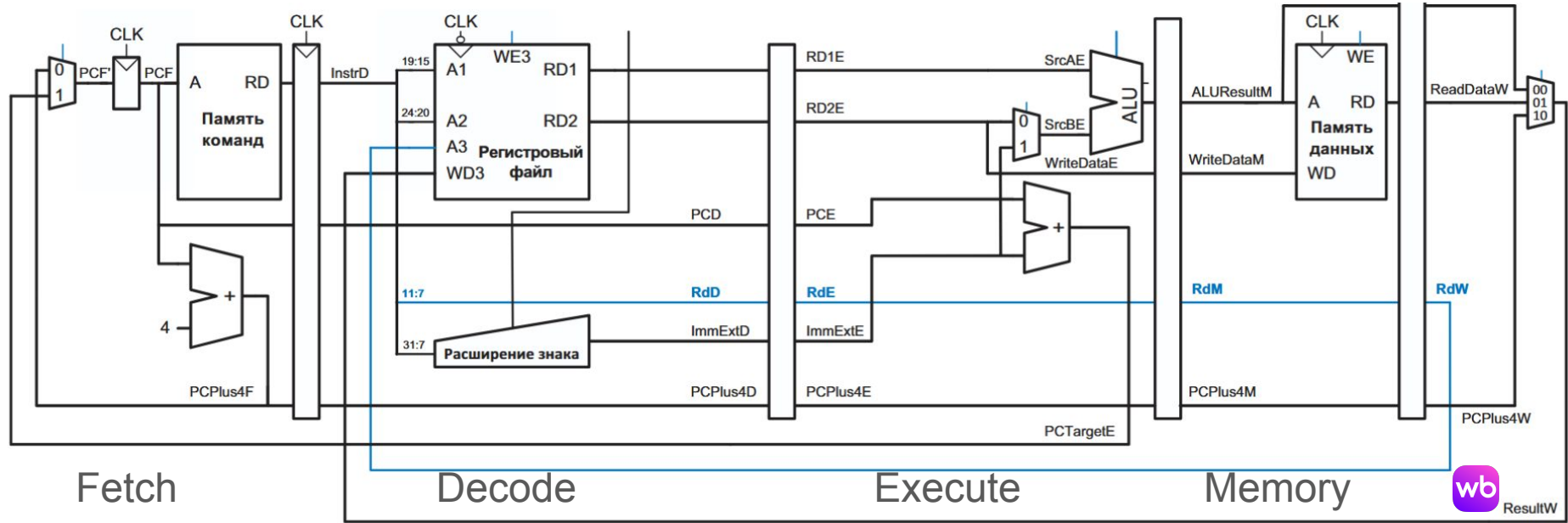
Pipelined CPU



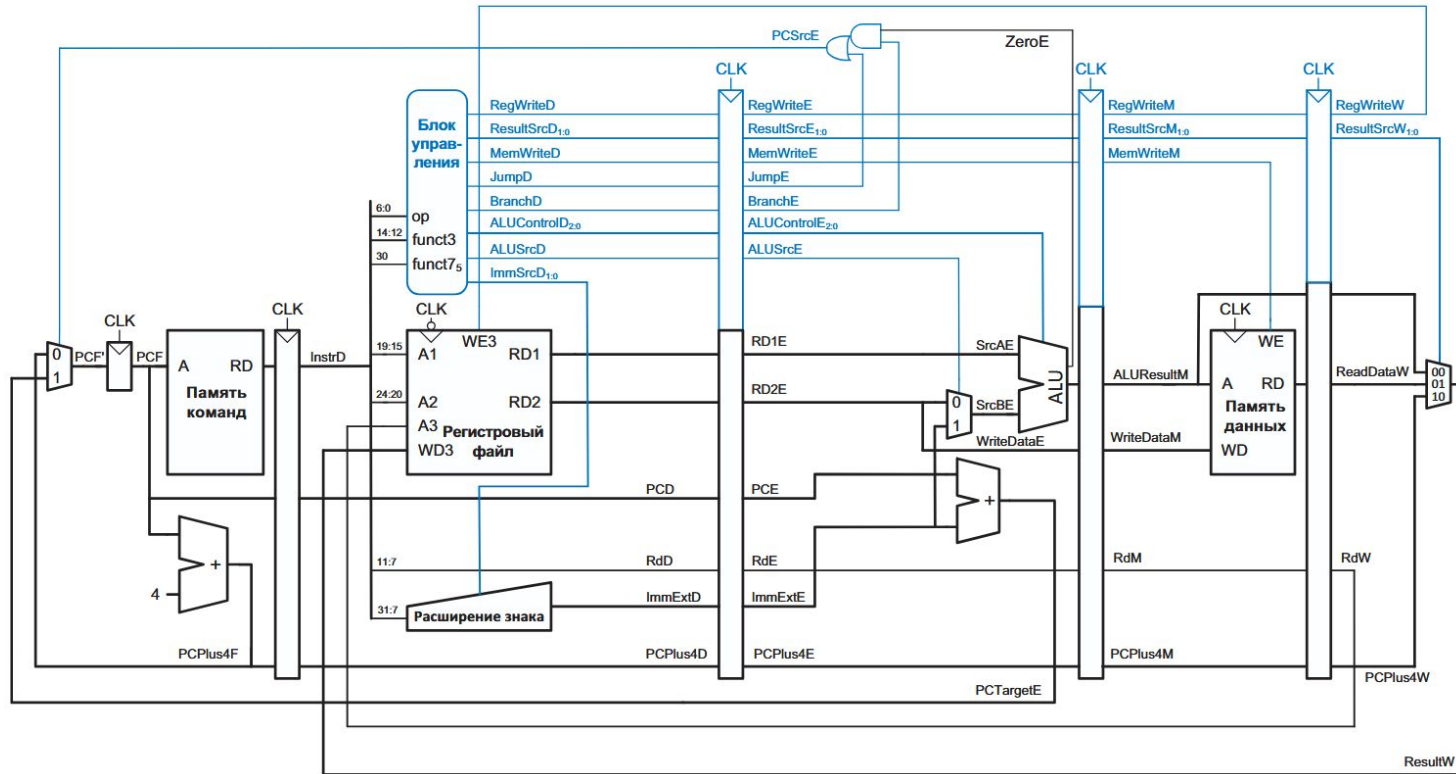
Single-cycle vs Pipelined CPU. Datapath



Pipelined CPU. Datapath

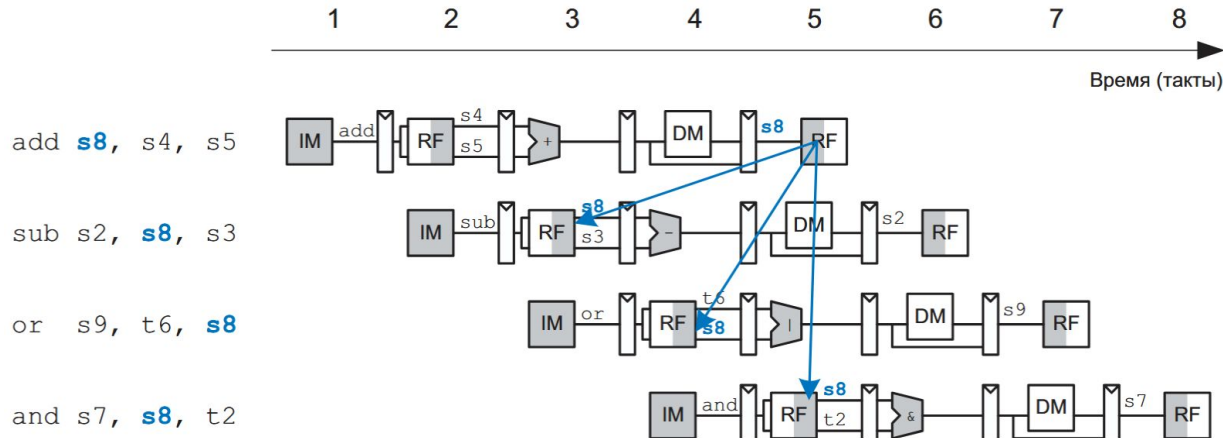


Pipelined CPU. Full Datapath



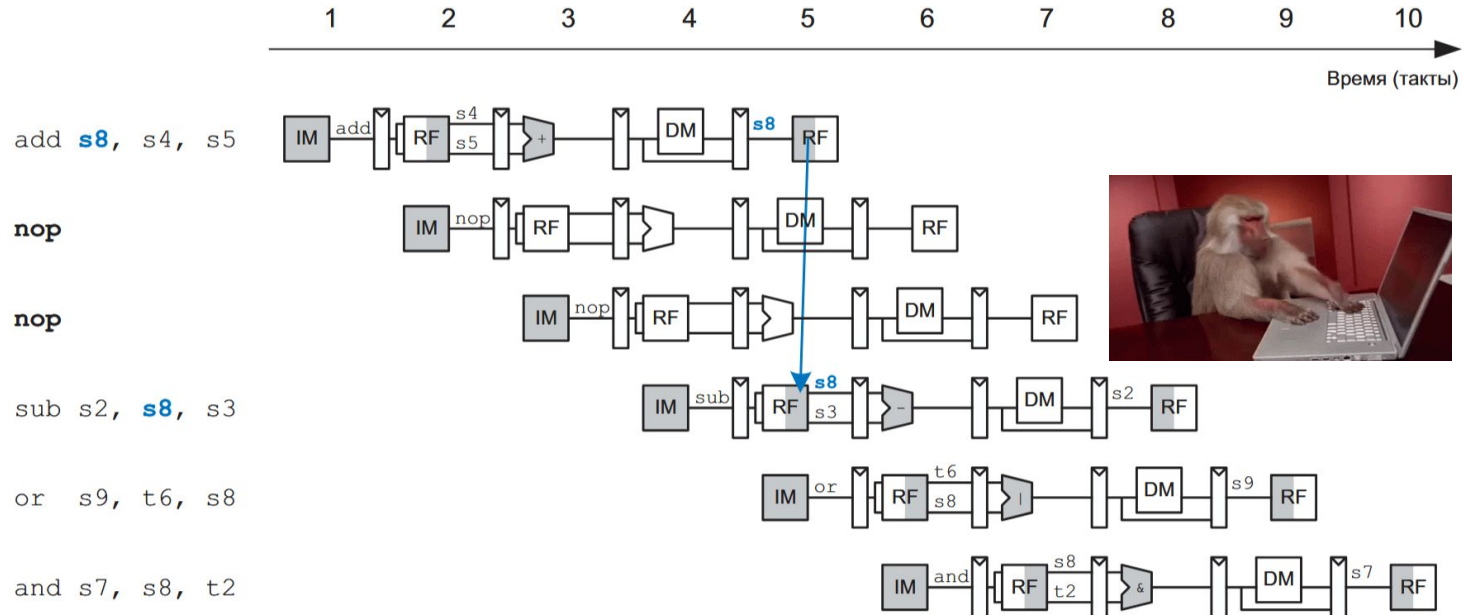
Pipelined CPU. Hazards

- Инструкция может зависеть от результата предыдущей, не завершённой инструкции
- Два вида конфликтов:
 - Data hazard - значение регистра еще не было записано в Regfile
 - Control hazard - следующая инструкция не известна (branch)



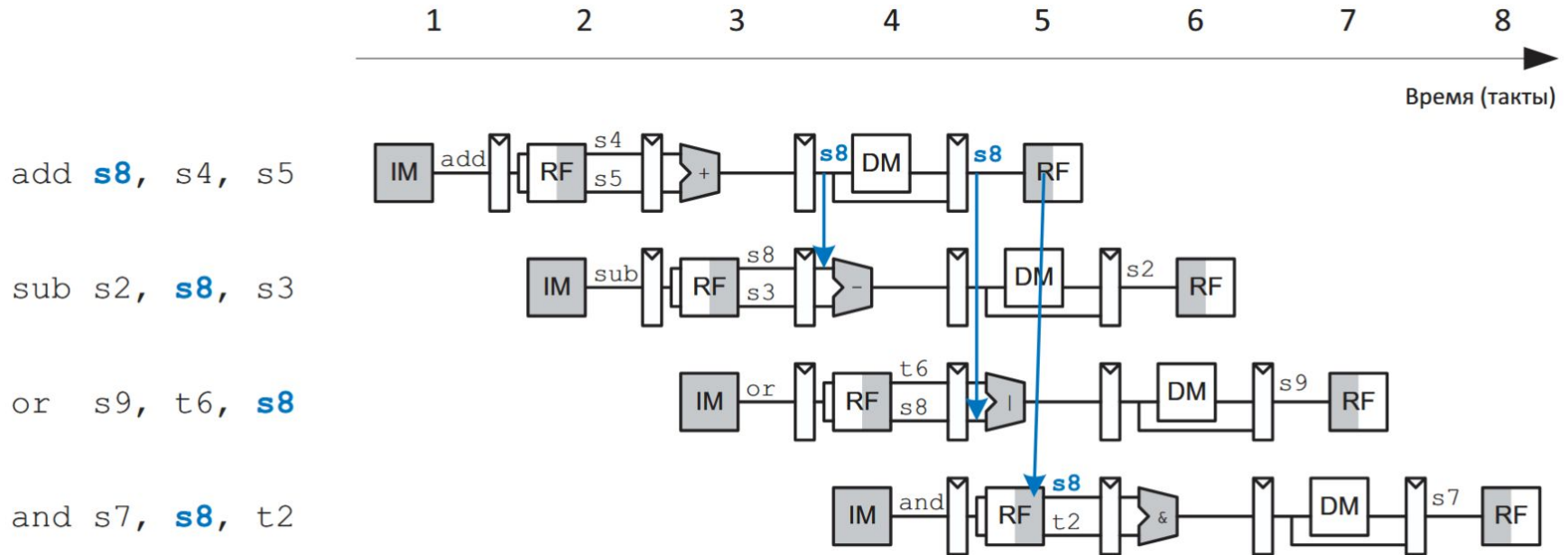
Pipelined CPU. Data hazards

- Простейшее решение - вставить nop (bubble)
- Кажется производительность ухудшилась

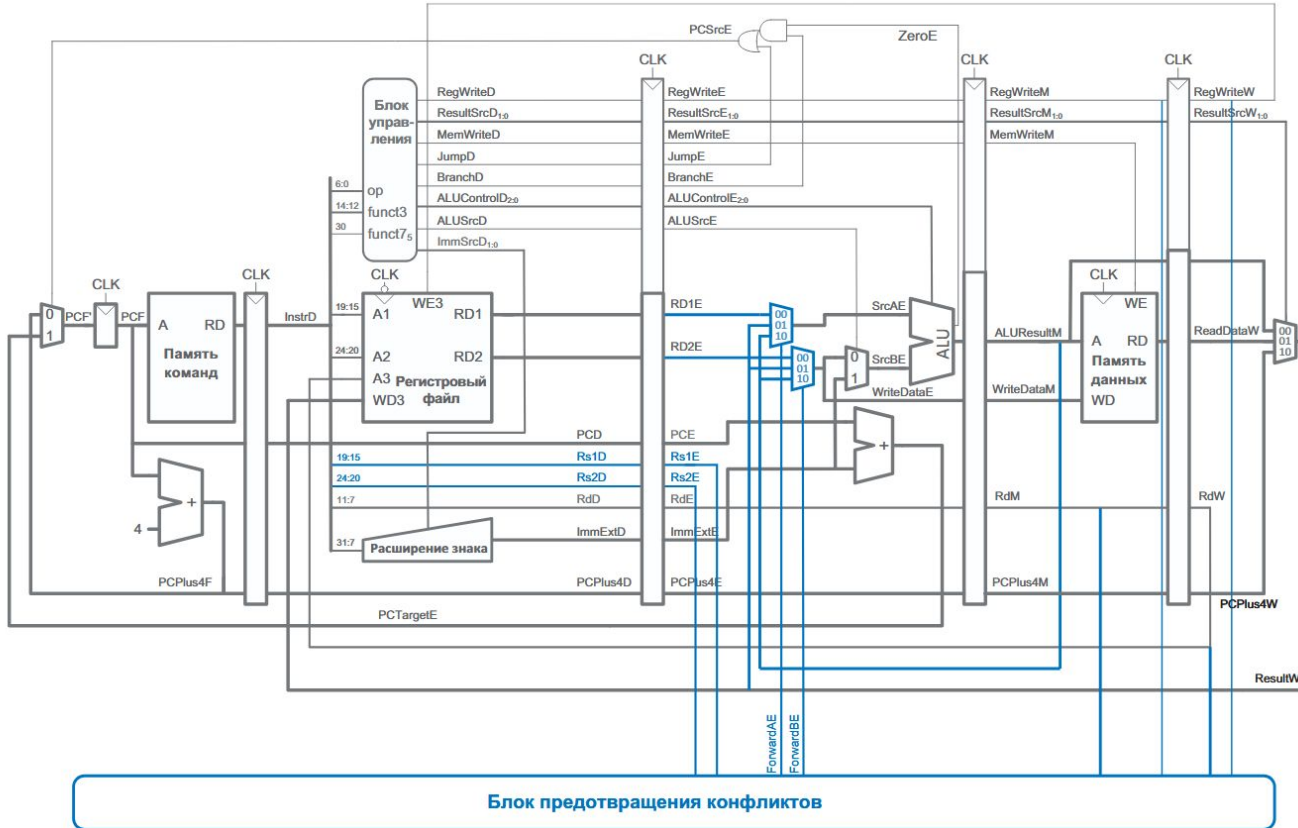


Pipelined CPU. Data hazards

- Будем пересылать данные как только они доступны (bypass)



Pipelined CPU. Hazard unit

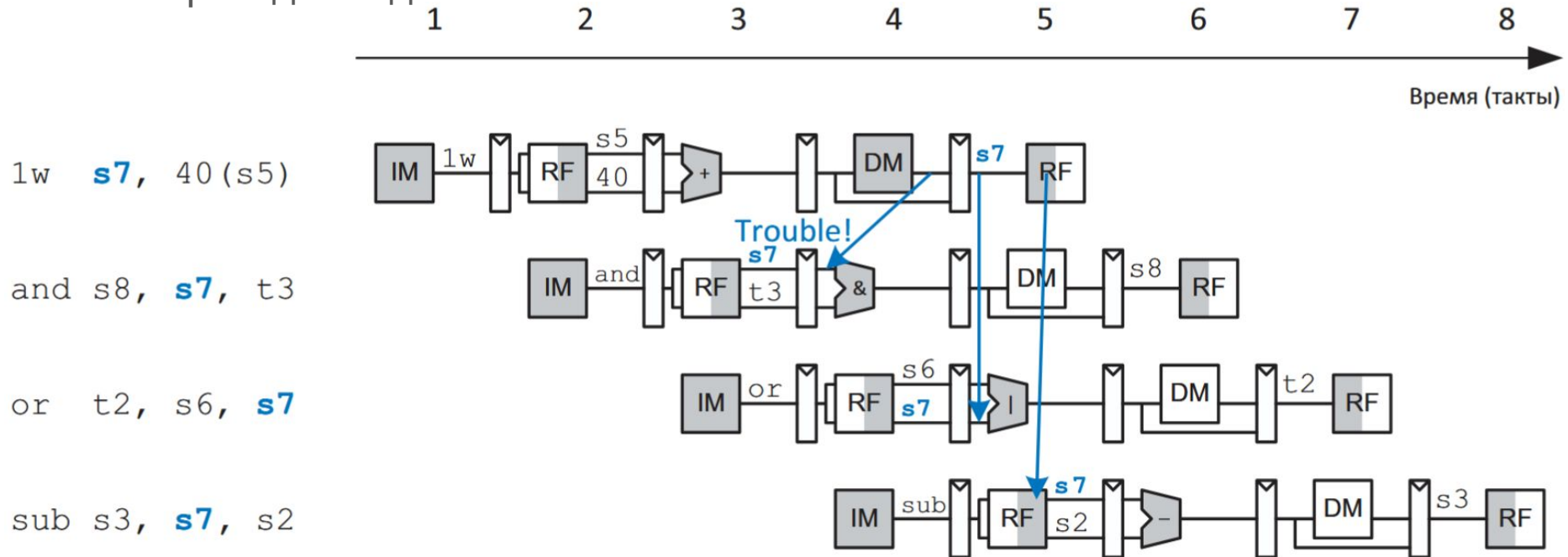


Pipelined CPU. Hazard unit



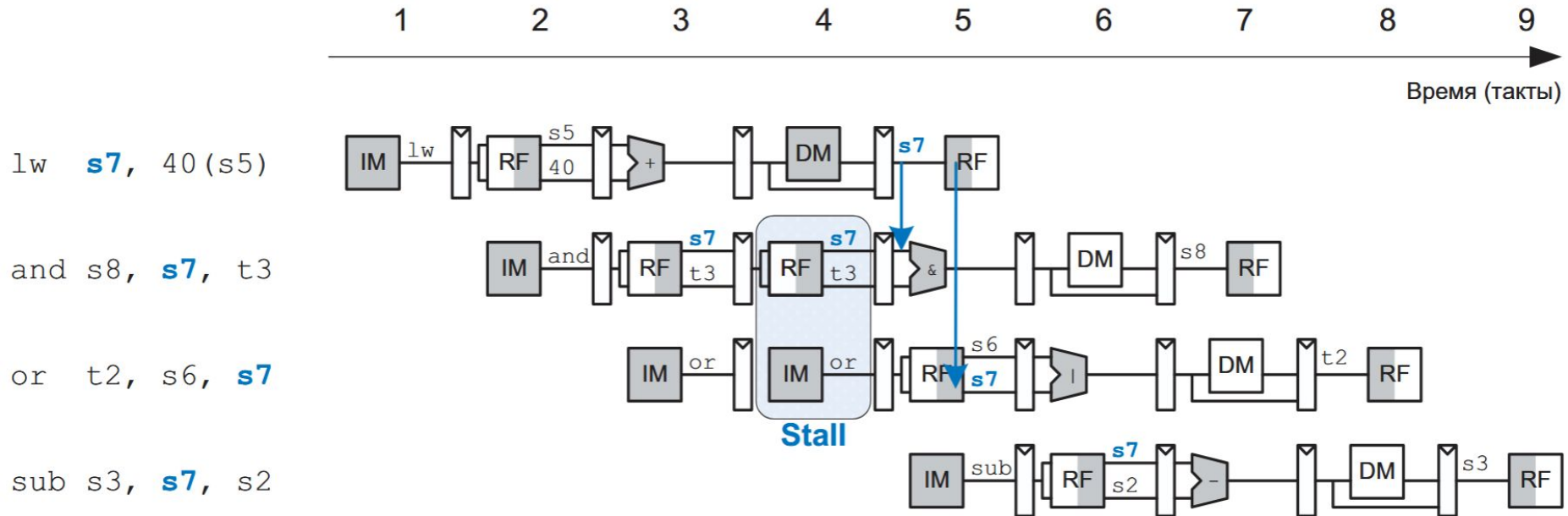
Pipelined CPU. lw problem

- lw не может сразу пробросить данные
- Приходится делать Stall

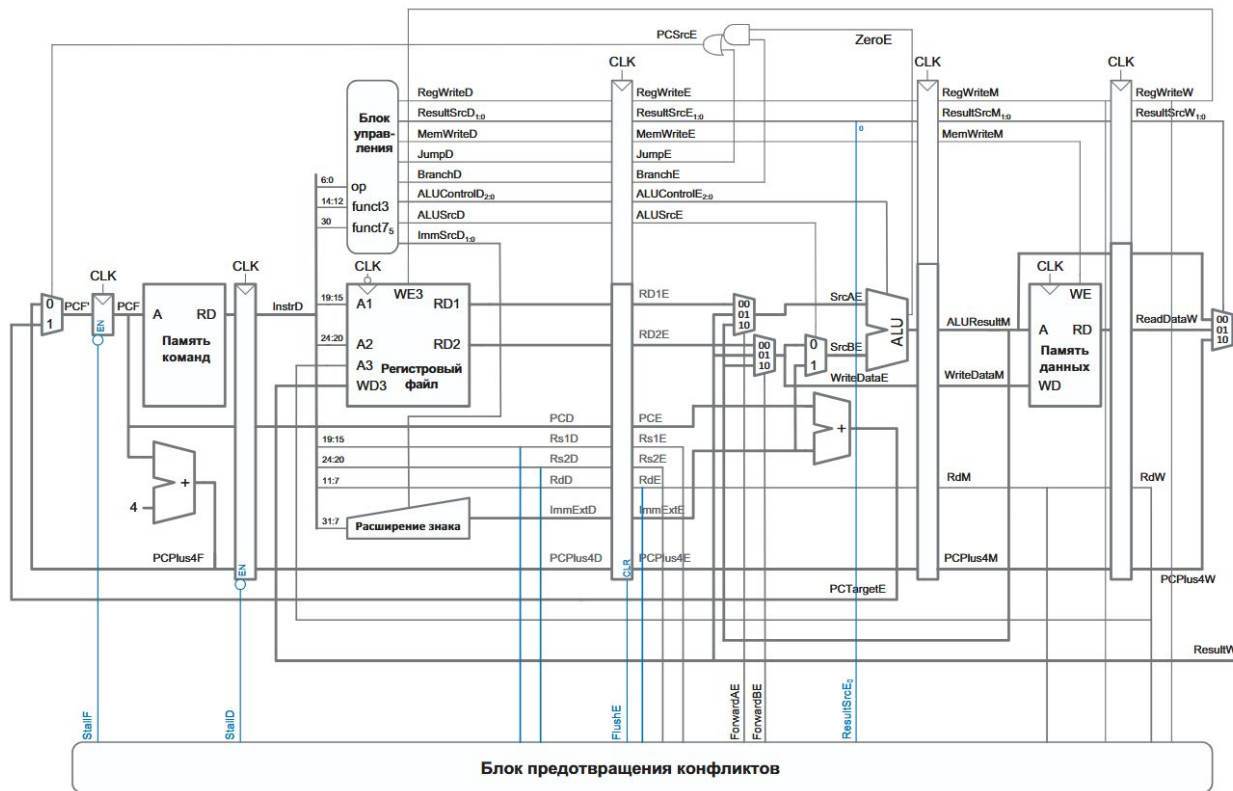


Pipelined CPU. lw problem

- lw не может сразу пробросить данные
- Приходится делать Stall



Pipelined CPU. Final (?) pipeline

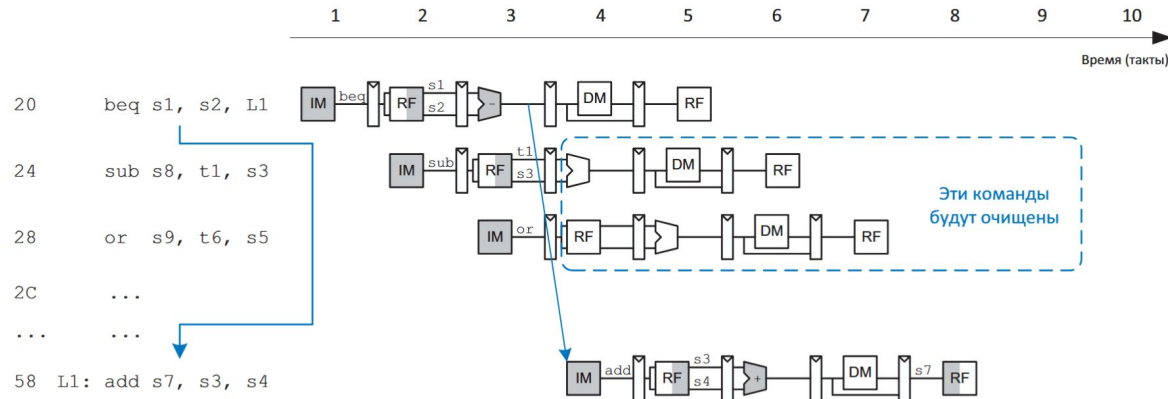


Pipelined CPU. Control hazards

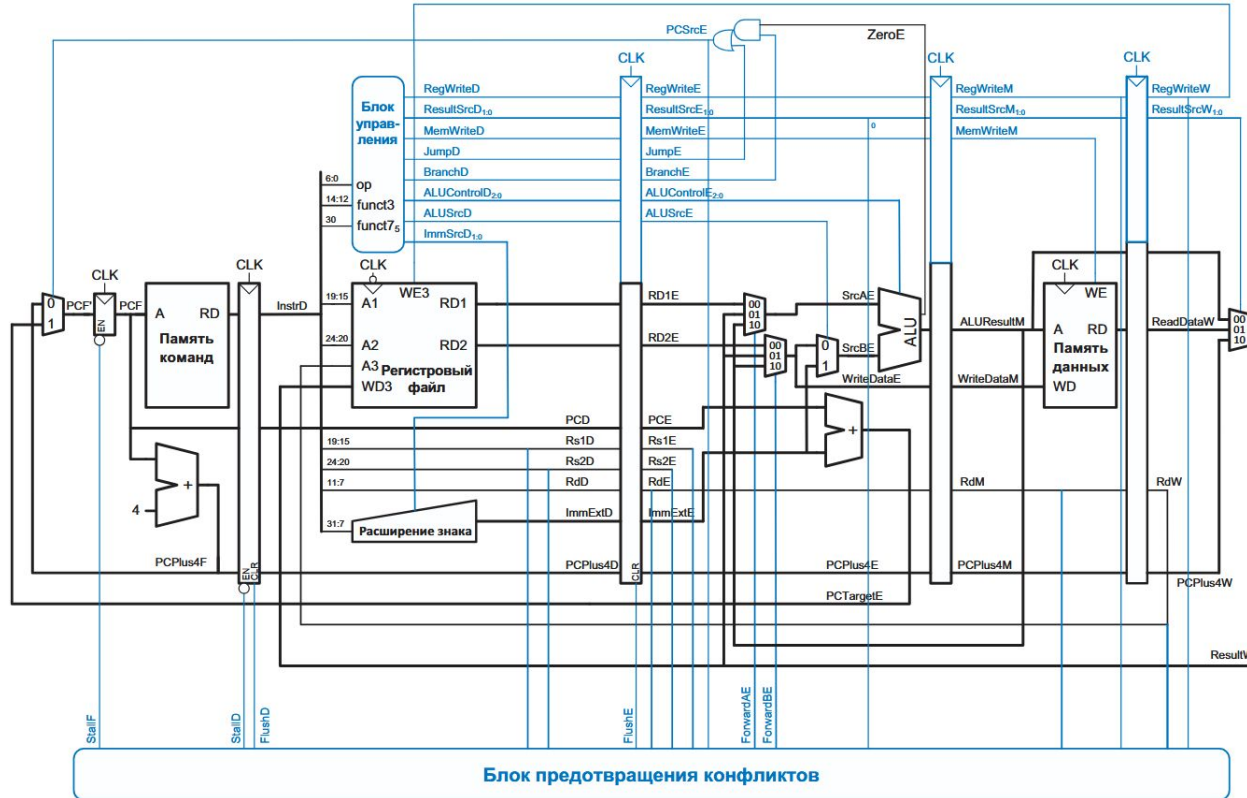
- beq:
 - Следующая инструкция известна **только** после Execute
- Простейшее решение - bubbles.
- Каждый бранч будет стоит 3 такта
- Бранчи весьма часто (20% инструкций)
- $CPI = 1 + 0.2 \times 3 = 1.6 \Rightarrow IPC = 0.63$
- Пропускная способность упала на **37%**

Pipelined CPU. Control hazards

- Решение получше - считаем что branch not taken
- Исполняем следующие инструкции
 - Если not taken - супер, никаких простоев
 - Если taken - выкидываем инструкции из взятого пути (pipeline flush)
- Если угадываем в 50% случаев
- $CPI = 1 + 0.2 * 0.5 * 3 = 1.3$, $IPC = 0.77$



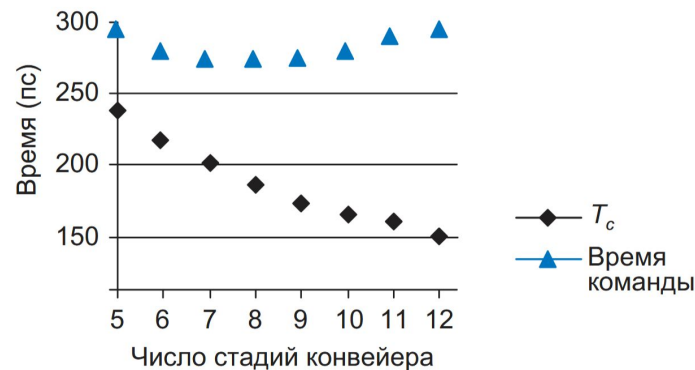
Pipelined CPU. Full hazard unit



Optimizations

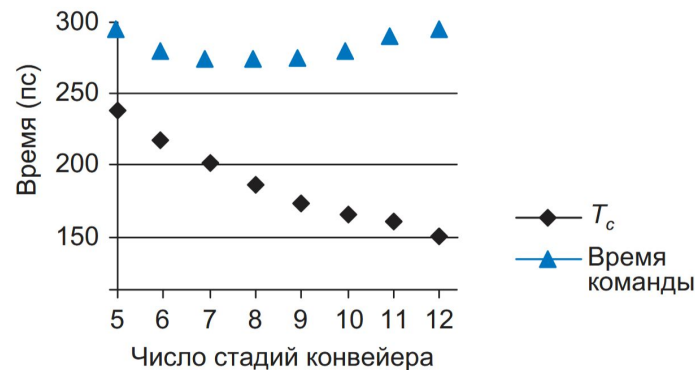
Deep pipelining

- Можно ли увеличивать стадии бесконечно?
- Обычно ограничиваются 10-20
- Ограничения
 - Конфликты
 - Расходы на поддержание конвейера
 - Мощность
 - \$\$\$



Deep pipelining

- Можно ли увеличивать стадии бесконечно?
- Обычно ограничиваются 10-20
- Ограничения
 - Конфликты
 - Расходы на поддержание конвейера
 - Мощность
 - \$\$\$



Branch prediction

- Мы уже изобрели тривиальный BP
- Два вида предикторов
 - Статические
 - backward = taken, forward - not taken
 - Динамические
 - Хранит историю бранчей в BTB (target + was taken)
- Чаще используются динамические, так как они опираются на историю исполнения бранчей
- Для оценки качества используется метрика mpki (mispredictions per kilo instructions) = $\text{misp_count} / \text{insn_count} * 1000$

Branch prediction. 1-bit

- Использует 1 бит для определения результата
- Слишком чувствительный

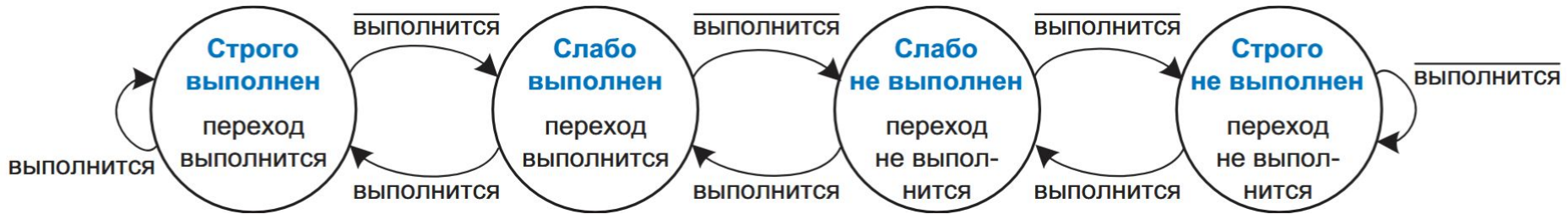
```
addi s1, zero, 0      # s1 = sum
addi s0, zero, 0      # s0 = i
addi t0, zero, 10     # t0 = 10
```

```
For:                  # for (i=0; i<10; i=i+1)
    bge s0, t0, Done
    add s1, s1, s0     # sum = sum + i
    addi s0, s0, 1     # i = i + 1
j      For
```

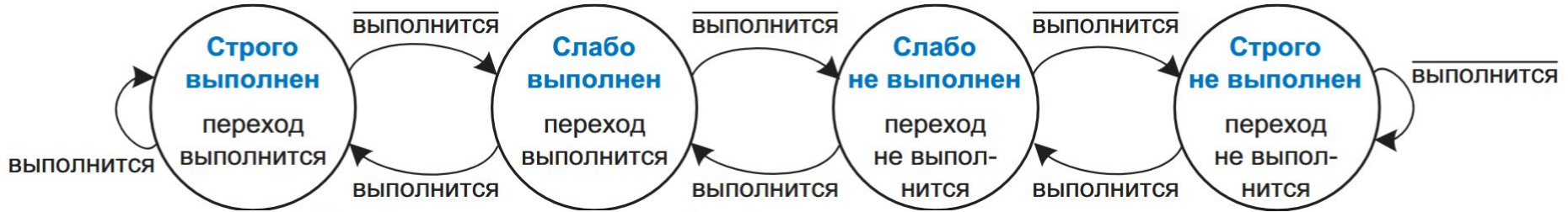
```
Done:
```

Branch prediction. Bimodal

- Используем два бита для более медленного изменения предсказания
- Используется как база почти во всех более сложных алгоритмах предсказания



Branch prediction. Bimodal



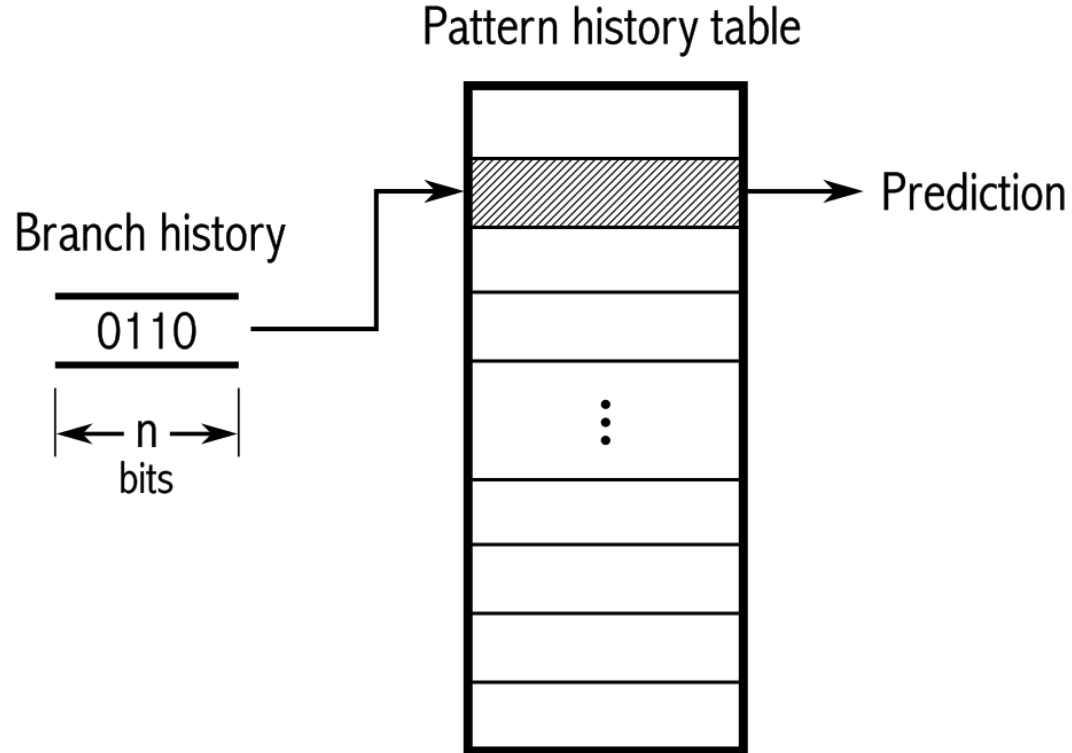
```
addi s1, zero, 0      # s1 = sum
addi s0, zero, 0      # s0 = i
addi t0, zero, 10     # t0 = 10
```

```
For:                                # for (i=0; i<10; i=i+1)
    bge s0, t0, Done
    add s1, s1, s0          # sum = sum + i
    addi s0, s0, 1          # i = i + 1
    j For
```

Done:

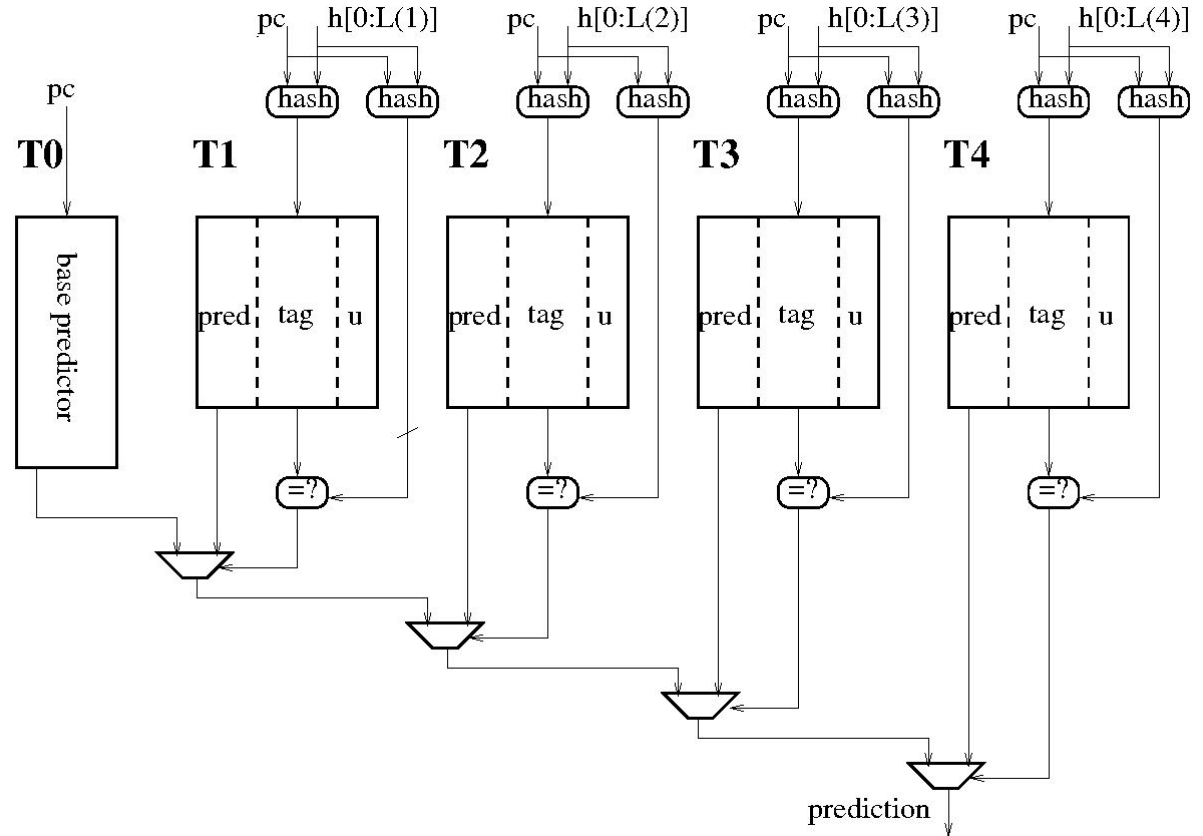
- . - .. - - . - .

Branch prediction. Two-level adaptive



Branch prediction. TAGE = Tagged + Geometric

- Свой “отрезок” глобальной истории
- Предсказание из самой *длинной* таблицы, где совпал тег

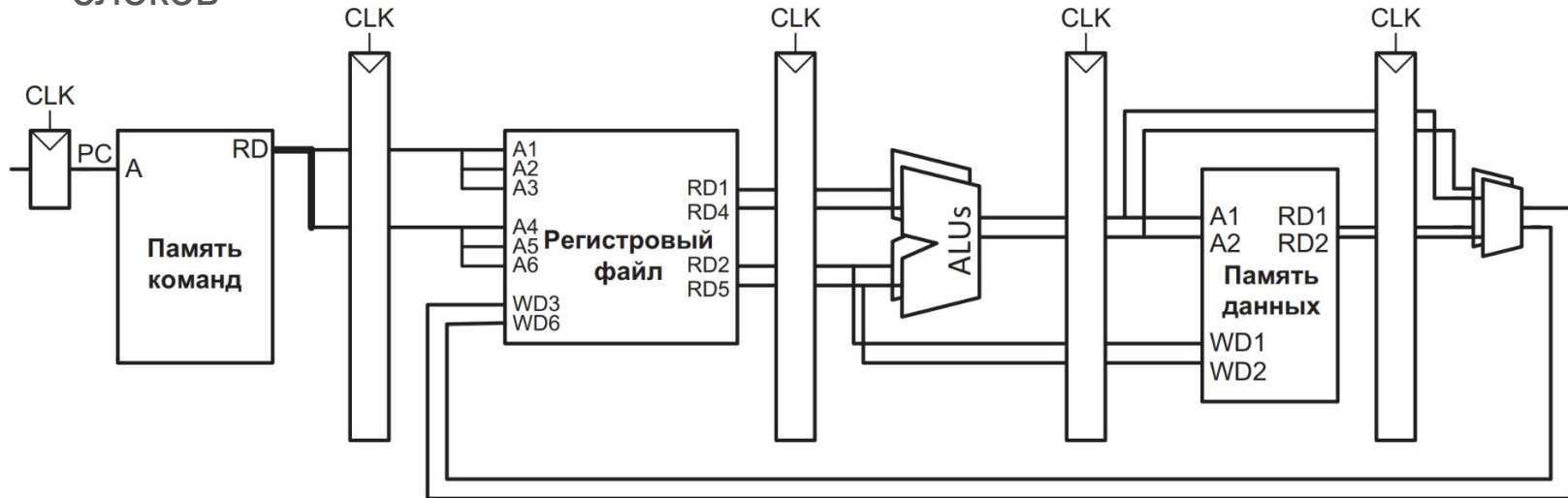


Loop prediction

- Любой branch predictor заточен на постоянство
- Если branch taken 10000 раз подряд, то с большой вероятностью предсказание сохранится
- Выход из цикла (даже постоянной длины) ведет к ошибке предсказания (misprediction)
- Для решения такой проблемы вводится Loop Predictor
- Предсказывает длину цикла, сообщая когда нужно изменить предсказание

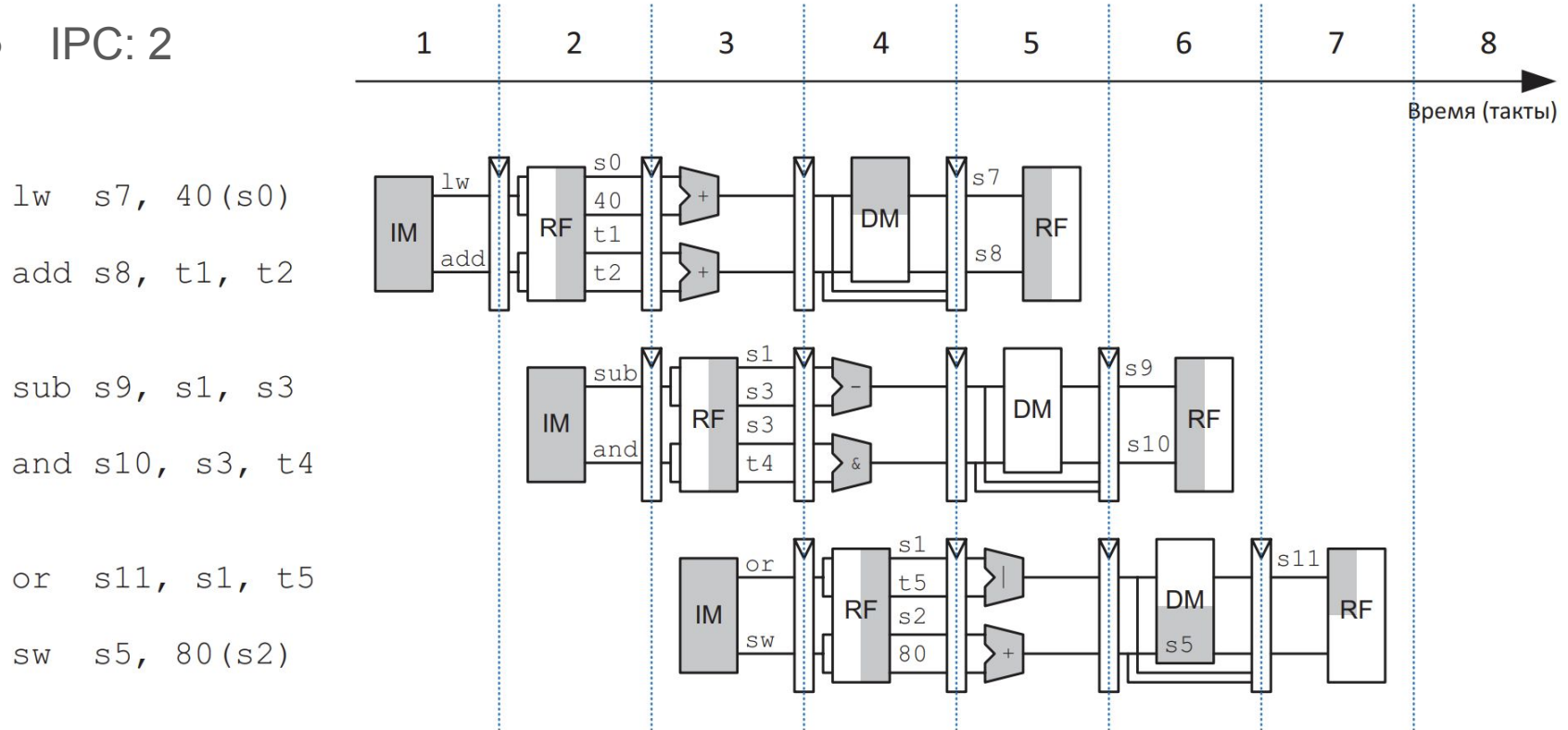
Superscalar Processors

- Зависимости затрудняют выполнение нескольких инструкций одновременно
- Предложение: тракт данных содержит несколько копий функциональных блоков



Example of Superscalar Processors

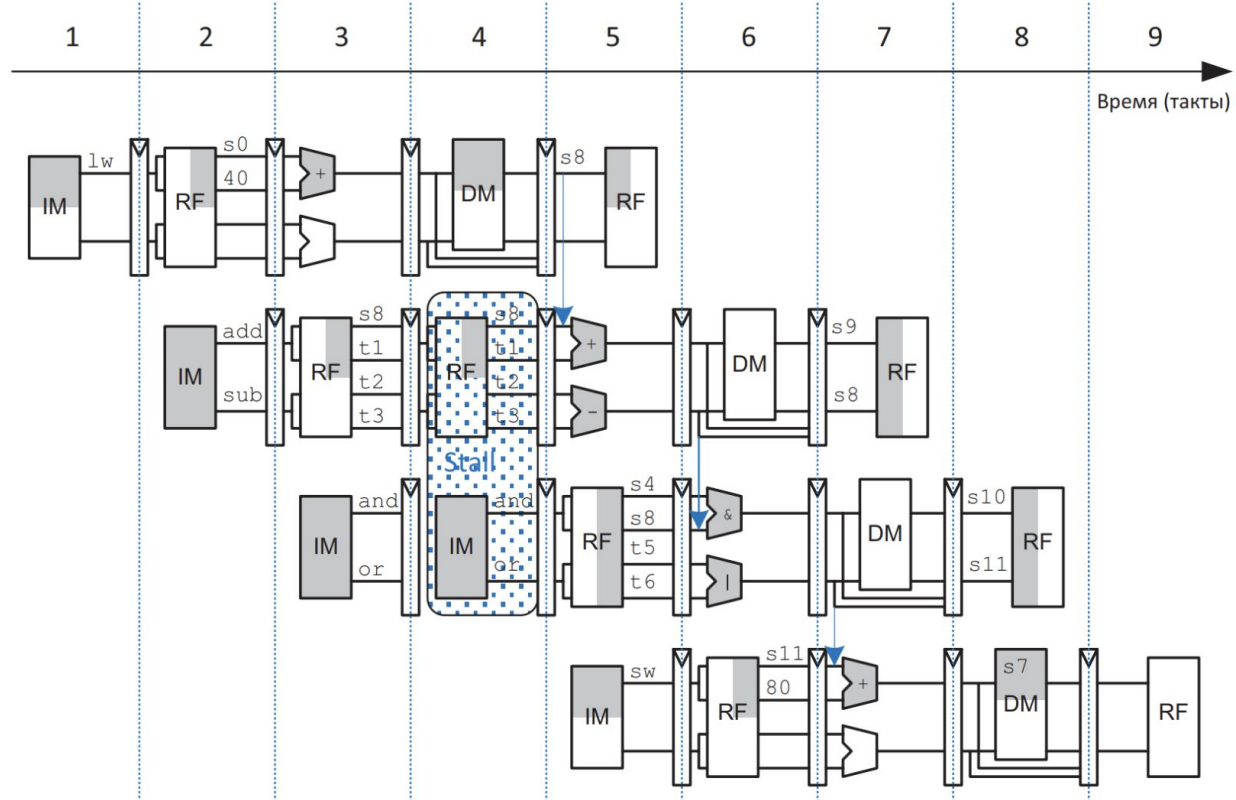
- IPC: 2



Superscalar Processors with Deps

- Идеальный IPC: 2
- Реальный IPC: 6/5

```
lw  s8, 40(s0)
add s9, s8, t1
sub s8, t2, t3
and s10, s4, s8
or  s11, t5, t6
sw  s7, 80(s11)
```



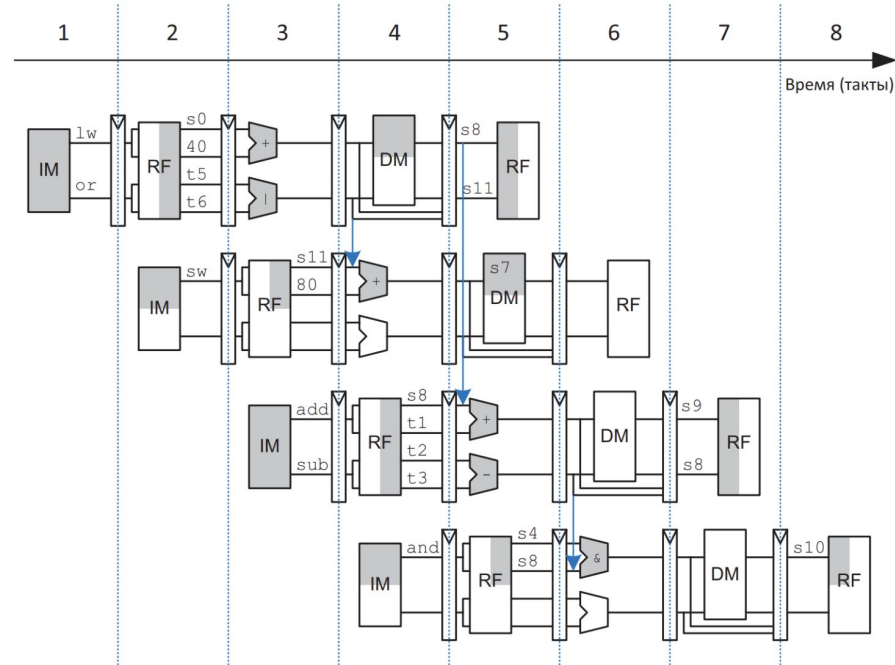
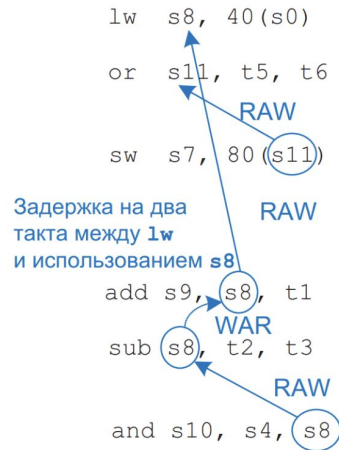
Dependencies Problem

- Есть запрос на следующее:
 - Просматривать несколько инструкций вперед
 - Выдавать (англ. issues) максимальное возможное количество инструкций одновременно
 - Выдавать инструкции в произвольном порядке (при условии отсутствия зависимостей)
- Виды зависимостей:
 - RAW (read after write): одна инструкция записывает, позже – инструкция считывает регистр
 - WAR (write after read): одна инструкция читает, позже – инструкция записывает в регистр
 - WAW (write after write): одна инструкция записывает, позже – инструкция записывает в регистр

Out of Order (OOO) Processor

- Процессор с внеочередным выполнением команд (O3 processor) заранее просматривает большое количество команд

- Идеальный IPC: 2
- Реальный IPC: 6/4



Register Renaming

- Чтобы устранить WAR-конфликты, процессоры с ОЗ используют прием, который называется переименованием регистров (register renaming)

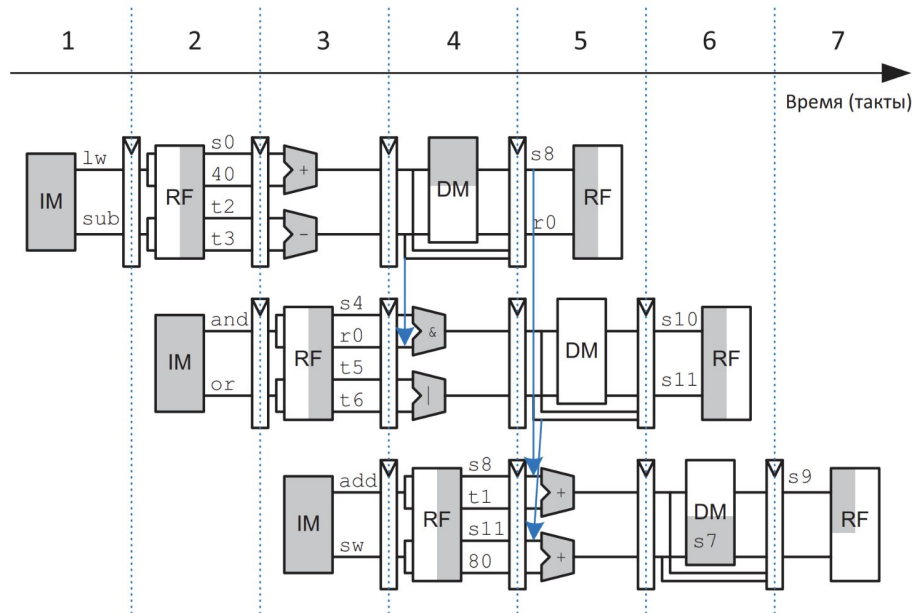
- Идеальный IPC: 2
- Реальный IPC: 6/3

```
lw  s8, 40(s0)
sub r0, t2, t3
and s10, s4, r0
or  s11, t5, t6
add s9, s8, t1
sw  s7, 80(s11)
```

RAW 2 такта

RAW

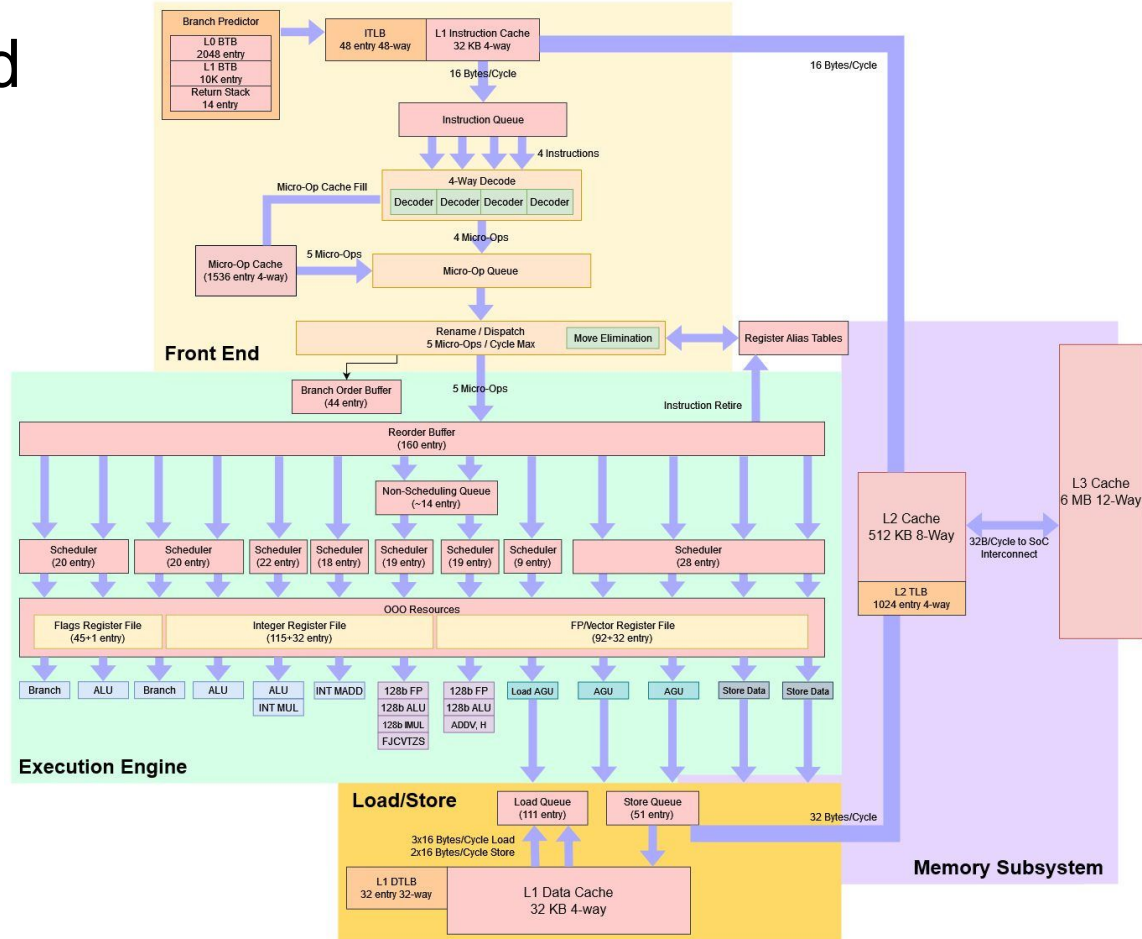
RAW



Real world

Cortex A710

Diagram By Clamchowder



Литература

1. [Основы цифровой электроники: учебное пособие](#)
2. [Цифровая схемотехника и архитектура компьютера, второе издание.](#)
[Дэвид М. Хэррис и Сара Л. Хэррис](#)
3. [NandGame](#)
4. [Turing Complete](#)